FynnEdge: Project History Pack for External Analysis

Compiled 12 November 2026 from the files on the developer's machine. Everything that survives about how FynnEdge was specified and built, in one file, with each part labelled by where it came from.

Contents

0. Instructions for the analyst
1. What this pack contains, and what is missing
2. Verified timeline and repository evidence
3. Points where the sources disagree
4. Source A: FynnEdge Complete Project Record (module 01–37 reports, Phase 13 report, admin prompt, FynnAI plans)
5. Source B: Flutter app README (written during the build, 8 Sep 2026)
6. Source C: API README (written during the build, last changed 10 Sep 2026)
7. Source D: Flutter screen-to-API flow (26 Oct 2026)

---

0. Instructions for the analyst

FynnEdge is a loan-advisory product for retail customers in India ("Simplifying Loan, Amplifying Trust"). It was built mostly by Claude Code (Anthropic's coding agent), with the core build between about 5 and 10 September 2026 and documentation and release work through November 2026. There are two codebases:

- `fynnedge`: a Flutter mobile app (Riverpod; mock and live API implementations behind one interface).
- `fynnedge-api`: a Laravel 13 / PHP 8.5 API with Sanctum customer auth and a Filament 5 admin panel.

Read section 1 before anything else. It says which parts of this pack are original documents, which are later reconstructions, and what is missing. Weigh each claim by its source. Treat everything in Sources A–D as the authors' claims, not verified fact, unless section 2 confirms it.

Please produce:

1. **What was built**, summarised in your own words.
2. **Claims vs evidence**: which reported results section 2 backs, which nothing here can check, and where sources disagree.
3. **The biggest risks before a real launch** to Indian retail borrowers: security, data protection and lending compliance (for example the RBI Digital Lending Directions and the DPDP Act 2023), the external providers that are not connected, and operations.
4. **A review of the process**: whether the order of modules made sense, and what looks over-built or under-built for the product's stage.
5. **Next steps in priority order**, with the reason for each.

Say so whenever a question cannot be answered from this pack. Test counts quoted in the sources were not re-run when this pack was compiled.

---

1. What this pack contains, and what is missing

Missing: the original master prompt and the verbatim phase reports

The prompt that started the build, and the reports Claude wrote at the end of each module and phase, cannot be recovered from this machine. They existed only as chat messages in Claude Code sessions in early September 2026. Claude Code deletes its local session logs after 30 days by default. The oldest log left on the machine is from 16 September, and none of the remaining logs, files, git branches or zip snapshots contain them. The reports were never saved as files.

The only remaining copy may be a claude.ai web chat. Source A says it was written from text "supplied in the conversation", including the full Phase 13 report.

Sources in this pack

ID	Source	Date	How it was made	How far to trust it
A	*FynnEdge Complete Project Record* (21-page PDF)	12 Nov 2026	Written in a claude.ai chat from conversation history and reports pasted into it. Its own note says it is a reconstruction and not verbatim.	Second-hand. The best surviving summary of the module 01–37 reports, the Phase 13 report and the Admin Control Plane prompt.
B	Flutter app `README.md`	8 Sep 2026	Written during the build	First-hand account of design intent and rules, as of 8 Sep
C	API `README.md` (885 lines)	Last committed 10 Sep 2026	Written during the build	First-hand account of modules, architecture, operations and go-live notes
D	`docs/flutter-screen-api-flow.md`	26 Oct 2026	Written by a later Claude Code session from the code	Reliable for the API surface as of 26 Oct
E	Section 2 of this pack	12 Nov 2026	Read directly from git and the running code	Verified facts, but narrow in scope

The API's `CLAUDE.md` / `AGENTS.md` is stock Laravel Boost setup text, not a project prompt, so it is left out.

Four separate numbering schemes

The sources use "module" and "phase" for four unrelated things:

Name	What it is	Where
Modules 01–37	Product build checkpoints, each with its own report and test run. These are the main "phase reports".	Source A §4, §4A
Phases 0–13	A Laravel/Filament conversion and admin integration run. Only the Phase 13 (final) report survives, as a summary. Phase 0 is the database backup `database.sqlite.phase0-20260910.backup`.	Source A §5
Admin Phases A–F	Steps *proposed* in a later Admin Control Plane prompt. Section 2 finds no evidence they were carried out.	Source A §6, Appendix A
"phase 1" (Flutter git)	A 10 Nov commit on the `production` branch that changes 2 lines in `android/app/build.gradle.kts`. It is release configuration, not a feature phase.	Section 2

2. Verified timeline and repository evidence

Checked on 12 Nov 2026 from git history, file timestamps and `php artisan route:list`.

Timeline

Date	What happened	Evidence
5 Sep	Flutter project scaffolded	File timestamps
7 Sep	First commits in both repos. Flutter `e5ad59e` "first flutter push": 563 files, +72,758 lines. API `628aa61` "first fluuter api module": 464 files, +83,486 lines. Code arrived in bulk, so it was written before the first commit.	git
7 Sep	Snapshot zips `fynnedge_mobile_sep7.zip`, `fynnedge-api_sep_7.zip`, `fynnedge-api_sep_7_1.zip`	File dates
8 Sep	Flutter `0c9e5e4` "updae sep 8": 369 files, +9,961 / −348. It adds support, decisions, intelligence/financial timeline, review and plan screens, which match modules 30–36 by name. Flutter README written.	git
10 Sep	API `cba74a1` "api main": 117 files, +15,471	git
10 Sep	API `3b335a8` "filament add on api" on `feature/sep_10_data`: 167 files, +11,171 / −1,853. Adds the Filament admin with 6 staff roles, `admin_users`, demo seeders and `demo:purge`, which matches the Phase 13 report. API README last changed here.	git; Source A §5
10 Sep	Phase 0 database backup and `fynnedge-api_sep_10.zip` snapshot	Files
13 Oct	Developer guide for a separate SMS OTP API (Areach provider). It belongs to the FynnEdge **website** project; the mobile API does not use it.	PDF in Downloads
26 Oct	API `65addde` "mobile api fultter": 10 files. Adds Scramble OpenAPI docs at `/docs/api`, a Postman collection and Source D. `main`, `test` and `production` all point here.	git
10 Nov	Flutter `b0ab705` "phase 1 fynnedge mobile api" on `production`: 2-line Android build change. APK build work.	git; session log
12 Nov	Source A generated; full snapshot `fynnedgecomp.zip`	Files

All 37 module checkpoints appear to come before the Phase 13 report: Module 37 reported 1,590 PHP tests, and Phase 13 reported 1,756.

Reported figures checked against the code

Claim	Source	Reported	Found on 12 Nov
Customer API routes	A §5	82	**82**, matches
Admin routes	A §5	33	**33**, matches
Total routes	A §5	129	**132**. The 3 extra are the OpenAPI docs routes added 26 Oct (`/docs/api`, `/docs/api.json`, `_scramble/dev-tools/devtools.js`).
Filament resources	A §5	14	**14**, matches

Claim	Source	Reported	Found on 12 Nov
Database	A §5	46 tables	41 migration files; table count not checked
Phase 13 changes left uncommitted	A §5	22 modified files + 13 new paths, not committed	Committed as `3b335a8` (167 files) on 10 Sep and now in `main`. Why 35 paths became 167 files is not explained.
Admin Control Plane deliverables	A App. A	`ADMIN_ARCHITECTURE.md`, `ADMIN_CAPABILITY_MATRIX.md`, `ADMIN_IMPLEMENTATION_REPORT.md`	None exist anywhere on the machine, and the Filament resource count is unchanged since Phase 13. No evidence Admin Phases A–F were done.
Test totals	A §4A, §5	PHP 1,756 (Phase 13); Flutter 1,282 + 113 skipped (Module 37)	**Not re-run.** 118 PHP test files and 49 Flutter test files exist.
Flutter `api_contract_test.dart` against a live server	A §5	Not run	Not run for this pack either
Mobile OTP delivery	C	`log` driver in development; a real SMS provider still to be chosen	Unchanged. No SMS provider is wired into the mobile API.

State of the repositories on 12 Nov

- **API:** `main`, `test` and `production` all point to `65addde`. The history is linear and the working tree is clean.
- **Flutter:** `main` and `test` point to `0c9e5e4`; `production` points to `b0ab705`. Uncommitted changes to `.gitignore`, `android/app/build.gradle.kts`, `AndroidManifest.xml`, `linux/flutter/generated_plugins.cmake` and `pubspec.lock` (APK release setup).

---

3. Points where the sources disagree

These are flagged for the analyst to weigh, not resolved here.

- **Admin roles.** Phase 13 reports six roles *built*: Super Admin, Business Head, Manager, Team Leader, Advisor, Viewer. The later Admin Control Plane prompt *proposes* a different six: Super Admin, Operations, Catalogue, Support, Compliance/Privacy, Read-only/Auditor. Source A §6 notes this.
- **Visual direction.** The app as built and golden-tested is a dark "financial cockpit". A later generated concept mockup is light blue and white, and was never implemented (Source A §3).
- **Phase 13 commit status.** See section 2.
- **Screen count.** Source A lists a 29-screen journey, and Source B says `screens_test.dart` renders all 29 screens. The app has 41 `*_screen.dart` files, because the 8 Sep commit added more screens.
- **External providers.** Every external provider is absent or fixture-only: SMS OTP, credit bureau, OCR, KYC, lender submission, provider webhooks, push, email/SMS/WhatsApp, advisor/CRM, malware scanning and LLM. Module 37's verdict is "engineering ready, external launch dependencies pending" (Source A §10–11; Source C "What is connected, and what is not").
- **Database concurrency.** Module 37 says the locking behaviour still has to be re-verified on the production database driver, because SQLite has no `SELECT … FOR UPDATE`. Phase 13 verified `migrate:fresh` on MySQL 8.4 but tested concurrency with only 2–8 local processes.
- **Retention periods** are deliberately left empty, pending a legal and product decision (Source A §10).

4. Source A: FynnEdge Complete Project Record

Text taken from `FynnEdge_Complete_Project_Record.pdf` (21 pages, generated 12 Nov 2026 in a claude.ai chat), with page headers and footers removed. **The PDF itself starts partway through its overview, at "Architectural rules"; sections 1–2 are missing from the original too.** Page 3 holds a concept mockup image that is not reproduced here; attach the PDF if you want it analysed.

> Architectural rules
> • Modular monolith; avoid premature microservices.
> • Business logic belongs in domain services/actions; controllers and Filament pages remain thin.
> • Flutter screen → controller/provider → repository/service → API. UI must not call HTTP directly.
> • Mock and real implementations sit behind the same interfaces; mock data must be clearly fictional.
> • Use one canonical calculation/service per domain concept; do not duplicate FinancialEngine, FynnScoreEngine, EMI calculation, affordability reference or recommendation ladder.
> • Model access must be mediated by a FynnAI gateway/provider interface; never expose Ollama directly to a mobile client.
> • LLMs must not receive unrestricted SQL, filesystem, shell or arbitrary web access.
> • Each consuming application gets its own identity and narrowly scoped tools.
> • Do not automatically train on customer data. Memory is not model training.
>
> API/mock seam
> The Flutter app uses FYNN_USE_MOCK and FYNN_API_BASE. UI remains independent of whether the repository is backed by mock data or the live API. Example live run configuration: flutter run --dart-define=FYNN_USE_MOCK=false --dart-define=FYNN_API_BASE=https://api.fynnedge.com.
>
> 3. Mobile application experience
> The original journey inventory covered 29 screens, with a premium financial-cockpit visual direction: dark cockpit surfaces, aqua-to-electric-blue brand gradient, ambient colour blooms, shared score dial, clean comparison table, opposed FynnMirror panels and a stepped conversational loan-discovery flow. The later generated visual mockup showed a lighter blue-and-white direction across eight sample screens; it is a concept mockup, not proof of implemented UI.
>
> | # | Screen / journey stage |
> |---|---|
> | 1 | Splash |
> | 2 | Welcome |
> | 3 | Mobile Login |
> | 4 | OTP |
> | 5 | Basic Profile |
> | 6 | Financial Intent |
> | 7 | Financial Goal |
> | 8 | Home Dashboard |
> | 9 | Explore |
> | 10 | Loan Discovery |
> | 11 | Loan Options |
> | 12 | Loan Detail |
> | 13 | Compare |
> | 14 | FynnTrust |
> | 15 | FynnMirror |
> | 16 | FynnLab |

17 FynnScore

18 FynnAI

19 Applications

20 Application Detail

21 FynnVault

22 Document Scan

23 Second Opinion

24 Profile

25 Financial Profile

26 Goals

27 Notifications

28 Privacy & Consent

29 Support / Advisor

Design implementation notes

- Bundled Manrope font; avoid runtime font fetching.
- Aqua-to-electric-blue brand gradient and restrained ambient colour blooms.
- Responsive checks at 360, 390, 420 and 430 px; golden tests in test/goldens/.
- FynnScore and FynnTrust share a signature score dial.
- Explore is a directory of hairline rows; Compare is a structured table; FynnMirror uses two opposed panels.
- Loan Discovery is conversational and stepped rather than a long undifferentiated form.
- Premium, modern, futuristic, trustworthy and minimal.

Generated visual concept

Eight-screen concept mockup generated during the conversation: Splash/Login, Dashboard, Loan Products, Loan Application,
Application Tracking, Financial Goals, FynnAI Chat, and Profile/Settings. Generated visuals can contain illustrative/sample content
and must not be treated as actual production data or a final approved design.

4. Product modules 1–37: implementation record

The following summaries consolidate the module reports supplied across the project discussion. “PASS” means the reported module checkpoint passed its stated tests; it does not imply that all external services are live or
that production deployment was independently verified.

01 — Customer Data Layer

users, customer_profiles, financial_profiles and financial_goals; decimal(14,2) money; server-derived financial
values; cross-field validation; ownership returns 404 for another customer; mock store supports goal mutation.

02 — Authentication

OTP challenges are bcrypt-hashed; previous live challenge retired; attempts increment before verification; random_int; production sender fails closed; Sanctum tokens with 30-day default expiry; per-number/IP rate limits; typed Flutter errors; global 401 handling; offline is not treated as signed out.

03 — Financial Profile

GET/PUT/PATCH financial profile and GET home; editable income, expenses, EMI, obligations, savings and employment type; server-authoritative validation; field-level errors; shared mock store; invalidation after save;
unsaved local derivations clearly labelled.

04 — Goals

GET/POST/list/detail/update/delete goals; customer ownership; saved amount cannot exceed target; past dates rejected; target_date uses YYYY-MM-DD; derived progress/months/monthly requirement; status read-only; no archive feature.

```
 05 — Financial Engine
 Pure shared calculation engine on Laravel and Flutter; integer paise precision; monthly outgo, surplus,
ratios,
 emergency months, goal metrics; definitions exposed by API; half-away-from-zero division; parity fixtures;
EMI
 via FinanceService.

 06 — FynnScore
 Versioned fynnscore-v1 with three equally weighted dimensions: EMI Burden, Spending Control, Emergency
 Buffer; bands Strong/Good/Needs Attention/At Risk; no score without income; goals excluded; no CIBIL,
 approval probability or income-stability factor.

 07 — FynnMatch
 Product matching by purpose/category, amount, tenure, published minimum income and age-at-maturity rules;
 affordability reference is caution, not automatic rejection; unavailable criteria do not pass; six
fictional sample
 products; no real provider connected.

 08 — Decision Experience
 Canonical ProductMatch across discovery/detail/compare/trust/twin/mirror/application; shared projection and
 45% affordability reference; comparison has no fabricated winner; simulation endpoint returns estimate;
state
 consistency and duplicate repository cleanup.

 09 — Application Journey
 Application states distinguish FynnEdge activity from provider decisions; immutable product/pricing
snapshot;
 reference FE-YYYY-NNNNNN; mock gateway only acknowledges receipt; idempotency and locking; no real
 lender submission; provider states modelled but unreachable until integration.

 10 — Privacy, Consent & Customer Controls
 Append-only, versioned consent catalogue; required service terms/privacy notice and optional AI memory;
 later consent types include credit bureau, lender sharing, AI training, marketing, document processing and
 identity verification; provider-gated availability; withdrawal/history; no legal wording fabricated.

 11 — FynnVault
 Private document storage; six document types; 10 MB limit; MIME/extension and server-detected validation;
 ULID storage keys; private authorized streaming; no storage path leakage; uploaded/failed states; no
 verification or sharing claim without real process.

 12 — FynnScan
 File picker supports PDF/JPG/JPEG/PNG/HEIC/WebP up to 10 MB; provider abstraction with unavailable and
 fixture providers; income family only; extraction fields employer, employee, gross/net income and pay
period;
 customer must review/confirm; scan API returns 501 when unconfigured.

 13 — FynnAI
 Deterministic assistant before external LLM integration; bounded authenticated customer context; memory
 only with consent and allowlisted preferences; no Vault content; provider absent returns 501; no language
 model in original implementation; rate-limited.

 14 — FynnTwin
 Three scenarios: take loan, change monthly spending, set aside monthly; reuses FinancialEngine,
 FinanceService, FynnScoreEngine and 45% reference; no persistence; 60-month cap for saving scenario;
 simulated score clearly distinct from real history.

 15 — FynnHome 2.0
 Removed fabricated insights/unread badges/hardcoded active application; Home reads canonical
 engine/score/goals; deterministic attention rules; onboarding profile persists; negative surplus shown as
 shortfall; customer-state journey sequence validated.

 16 — Decision Loop / UX Audit
 Navigation and stale flows cleaned; canonical journey Home → Goal → Explore → Discovery → Options → Detail
 → Trust → Twin → Mirror → Apply → Application → Home; same EMI verified through journey; state invalidation
 and terminology aligned.

 17 — Production Hardening & Trust Audit
 Critical/high findings zero at that checkpoint; demo seeder guarded; Vault write limiter; privacy/script
checks;
 127 IDOR assertions; tamper and concurrency tests; production config examples; external infrastructure
still
 needs deployment verification.

 18 — Customer Experience Validation
```

Customer journey scorecard; fixes to decline claims, discovery wording, FynnScore definition, affordability terminology, application prominence, match badge meaning, PAN implication, optional city and age explanations; walkthroughs validated.

19 — Data & Product Reality Audit
Provenance map; unsupported cheapest/approval-time claims removed; malformed product validation; missing eligibility rules are unavailable, not passed; sample catalogue marked fictional; PASS.

20 — Production Product Catalogue Architecture
Provider → CatalogueProduct → ProductVersion → ProductSource → resolved LoanProduct → ProductRepository → FynnMatch and downstream journey; immutable version/provenance architecture; no real provider.

21 — Product Administration & Catalogue Operations
Persisted providers, catalogue_products, product_versions and append-only catalogue audit events; create/draft/validate/publish/activate workflow; published versions immutable; initial CLI administration; customer read isolation.

22 — Provider Integration & Catalogue Ingestion
ProviderAdapter → ProviderFeed → Normalizer → CanonicalProductInput → CatalogueService → ProductVersion; fixture provider only; idempotent fingerprinting; all-or-nothing feed; draft by default; no sample fallback.

23 — Credit Bureau / Credit Intelligence
No live bureau connected; driver none; fixture-only bureau data; consent and adapter boundary; credit_reports; only supported min_credit_score criterion; CreditProfile separate from FynnScore; unavailable
does not become pass.

24 — FynnScan Real OCR & Document Intelligence
No live document provider; fixture adapter; document_processing consent; income-only extraction; immutable extraction and review records; customer review required; no raw provider payload/text/image storage; queued processing and malware scanning remain external prerequisites.

25 — KYC & Document Verification
No live KYC provider; fixture provider only; Aadhaar boundary deliberately contains no UIDAI shortcut; separate
identity_verification consent; no automatic profile mutation; no generic verified=true; canonical data minimisation; raw provider response not stored.

26 — Real Lender Application Submission
No live lender provider; fixture provider only; ApplicationStatus separate from SubmissionOutcome; unknown outcome requires reconciliation and cannot retry; explicit minimal field map; lender_data_sharing consent; no
sensitive score/document/AI data sent; strict idempotency and concurrency.

27 — Application Document Journey
Application requirements derived from snapshotted product category; application_documents tracks requirement selection and review; choosing is not confirming; server-authoritative checklist; deleted/stale file
becomes missing/needs_review; no lender document transmission.

28 — Application Status & Provider Events
No live provider event source; authenticated provider ingress; provider_events stores hashes/metadata, not raw payload; idempotency; stale/conflicting events ignored; application status, submission outcome and provider event remain separate; unknown reconciliation only from valid provider event.

29 — Notifications & Customer Communication
No real push/FCM/APNs/email/SMS/WhatsApp; notification policy is sole copy/decision authority; notification creation distinct from delivery; concurrency-safe creation; sensitive content guard; no marketing implementation; no notification preferences without provider.

30 — Support / Advisor Experience
No live advisor/CRM/chat/email/SMS/WhatsApp; support_cases/messages/events; controlled states and context types; ownership checks; Flutter support journey; advisor author type exists but no advisor write path; no support agent can alter financial outcome.

31 — Financial Intelligence & Personalisation
Deterministic observations, moments, insights and next-best-action; no LLM required; no invented trend; no score-history reconstruction; memory bounded; personalisation changes presentation/order/action only, never eligibility/pricing/score/provider outcome.

32 — Decision History & Explainability
Records real customer simulation and comparison actions; history reads actual application events; immutable snapshots and provenance; no inferred page views or invented customer decisions; historical values remain

```
as
 they were at decision time.

 33 — Financial Coach & Action Planning
 Extends the existing deterministic NextActionRules ladder; one source of truth for Home and plan; shortfall
 suppresses borrowing exploration; no new table/endpoint/repository; no fabricated advice or LLM dependency.

 34 — Financial Plan & Goal Execution
 Plan is a derived reading, not a stored record; reuses FinancialPosition, score, goals and action ladder;
no
 separate engine/table/endpoint; safety wording avoids commitments/guarantees/on-track claims; no
 primary-goal concept.

 35 — Financial Progress & History
 Uses existing financial_observations; compares consecutive stored observations only; no current-vs-history
 false comparison; dedup via digest/follows; unchanged saves skipped; historical score and ratios immutable;
 no chart or goal history.

 36 — Financial Review & Periodic Checkup
 Review is a reading of intelligence; no review table/schedule/reminder/score; consistent score/goals/action
 ladder; side-effect-free reads; no review-completion fiction; safety language avoids forecasts and
 commitments.

 37 — Production Readiness & Final System Audit
 No new feature; architecture/security/IDOR/auth/privacy/financial
correctness/API/application/document/intellig
 ence/concurrency/rate-limit/performance/operations/Flutter/provider reality audited; zero P0; external
 dependencies remain; engineering ready, external launch dependencies pending.

 4A. Reported verification milestones
 Test counts below are point-in-time reports from different modules/checkpoints. They are not
 additive and should not be interpreted as one cumulative test count. Later reports supersede
 earlier counts for the same suite.

   Checkpoint               PHP tests / assertions                Flutter tests         Live API / other

   Module 17                Critical/high audit; 127 IDOR         —                     Production
hardening checkpoint
                            assertions

   Module 24                909 tests; 5,484 assertions           919 + 73 skipped      73 live

   Module 26                1,036 passed; 6,683 assertions        968 + 77 skipped      77 live

   Module 28                1,095 passed; 7,279 assertions        998 + 82 skipped      82 live

   Operational              1,145 passed; 7,534 assertions; 0     998 + 82 skipped      82 live
   hardening                risky

   Canonical adapter        1,190 passed; 7,626 assertions; 0     998 + 82 skipped      82 live
   boundaries               risky

   Module 29                1,293 passed; 8,064 assertions; 0     1,027 + 88 skipped    88 live
                            risky

   Module 30                1,341 passed; 8,349 assertions; 0     1,072 + 95 skipped    95 live
                            risky

   Module 31                1,404 passed; 8,988 assertions; 0     1,101 + 101 skipped   101 live
                            risky

   Module 32                1,404 passed; 8,988 assertions        1,101 + 101 skipped   101 live

   Module 34                1,504 passed; 10,155 assertions;      1,206 + 111 skipped   111 live
                            0 risky

   Module 35                1,541 passed; 10,514 assertions;      1,234 + 112 skipped   112 live
                            0 risky

   Module 36                1,561 passed; 11,031 assertions;      1,282 + 113 skipped   113 live
                            0 risky

   Module 37 final          1,590 passed; 3 skipped; 11,120       1,282 + 113 skipped   113 live;
Analyze/Pint clean
```

```
                            assertions; 0 risky
 Module 37 also reported 77 goldens at 390 px, 35 at 420 px and 21 at 360 px. Three PHP skips required
pcntl;
 race behaviour was separately exercised manually with parallel processes.

 5. Phase 13 — Laravel / Filament conversion
 report
 The supplied Phase 13 report describes a conversion/administrative integration checkpoint. It
 is distinct from the 37 product modules above. The report states that the conversion finished
 successfully while explicitly retaining production and external-provider limitations.

   Area                          Reported final state

   Framework                     Laravel 13.30.1; PHP 8.5.4; Composer 2.9.5

   Admin framework               Filament 5.8.1

   Customer API auth             Sanctum 4.3.3

   Database                      46 tables; 502 columns; 41 foreign keys; 130 indexes (78 unique)

   Migrations                    41 → 46 tables; migrate:fresh verified on MySQL 8.4 and SQLite

   Seeders                       4: one default, three opt-in; all refuse in production

   Foreign keys                  0 orphans across 41 foreign keys

   Application rows              Original 6; current 58; 52-row gain is labelled demo fixtures

   Schema comparison             36 tables compared; 36 matching; 0 differing; 0 missing

   Original SQLite               Byte-identical to Phase 0 checksum; off-repo backup also unchanged

   API routes                    82 original and 82 final; 82 preserved; 0 added/changed/removed

   Total routes                  129 = 82 API + 33 admin + 14 framework/Livewire

   Filament                      14 resources; 33 routes/pages; 2 widgets; 9 navigation groups

   Admin roles                   6: Super Admin, Business Head, Manager, Team Leader, Advisor, Viewer

   Tests                         1,756 tests; 1,753 passed; 3 skipped; 0 failed; 11,702 assertions

 Data integrity and compatibility
   • All 82 API routes retained identical URI, HTTP method, controller action, auth:sanctum binding and
throttle name
     according to the report.
   • Flutter compatibility: 82/82 endpoints exercised live; OTP → token → me → logout verified; response
data/message
     envelope unchanged; ownership 404-not-403 verified across five customers.
   • Sanctum token hashes are SHA-256 only; 30-day expiry; per-device revocation.
   • Separate admin guard and admin_users table/session/password; customer token cannot access /admin; admin
     session cannot access /api/v1; AdminUser has no createToken() method.
   • Three behaviour fixes: customer consent scope no longer leaks across customers; concurrent first sign-
in
     succeeds; dev_hint requires both non-production and FYNNEDGE_OTP_DEV_HINT.
   • Demo cleanup command php artisan demo:purge supports reserved mobile prefix, cascade preview and
     confirmation; scratch test preserved the real seeded customer.

 Security findings
   Severity                        Count / disposition

   Critical                        0

   High                            0

   Medium                          3

   Low                             4

   Fixed                           6 findings

   Accepted with rationale         1: wildcard token abilities accepted for one client where no enforcement
point exists
```

```
 Explicitly unverified / production work
   • Credit checks, lender submission, KYC, document scanning and push: drivers are none; refusal paths
tested (501
     with machine-readable reason); success paths not verified.
   • Flutter app’s own api_contract_test.dart against live server was not run; the server-side contract was
exercised.
   • Production configuration was not run with APP_ENV=production.
   • Concurrency was tested with 2–8 local parallel processes, not production traffic.
   • Filament login was not driven through a real browser session; guard credentials and authenticated page
rendering
     were tested.
   • Before production: APP_DEBUG=false; APP_ENV=production; LOG_LEVEL=info; SESSION_ENCRYPT=true;
     SESSION_SECURE_COOKIE=true; configure TRUSTED_PROXIES; unset FYNNEDGE_OTP_DEV_HINT; purge demo
     records/admin; set real FYNNEDGE_ADMIN_EMAIL/PASSWORD; configure real OTP driver; run Flutter contract
test;
     decide collation changes only deliberately.

 Working-tree status
 The report says nothing was committed: 22 modified files and 13 new paths remained in the
 working tree on branch feature/sep_10_data, for owner review. Do not assume the changes
 have been committed or deployed.

 6. Admin Control Plane — requirements and
 governance
 The requested Admin Panel is the single operational control surface for FynnEdge. It should
 provide full visibility and legitimate operational/configuration control across all 37 modules,
 but it must not become a backdoor around domain integrity.

 Definition of full control
 Full control means: administrators can observe everything relevant, configure what is configurable, operate
 what is operationally controllable, manage providers/catalogue/support, perform approved corrections, audit
 activity, and see why an action is blocked. It does not mean arbitrary database edits or authority to
fabricate
 financial/provider outcomes.

 Recommended navigation
   Area                       Sections

   Dashboard                  Platform KPIs; financial intelligence; operations; provider reality

   Customers                  Customers; profiles; financial profiles; goals; documents; consents

   Decisions                  FynnScore; FynnMatch; FynnTrust; FynnTwin; Intelligence; Coach; Plan;
Progress; Review; Decision
                              History

   Applications               All applications; documents; submissions; provider events; unknown outcomes

   Catalogue                  Providers; products; product versions; sources; ingestion; publication

   AI & Intelligence          Existing FynnAI status/configuration; models/agents/tools only where
implemented

   Operations                 Notifications; support; queues; health; retention; malware scanning

   Configuration              Financial rules; feature flags; provider configuration; consent versions;
security; settings

   Governance                 Admin users; roles/permissions; audit logs; data operations

 Admin roles discussed
   Role                              Intended scope

   Super Admin                       Full admin governance and configuration

   Operations Admin                  Applications, documents, provider events, notifications, support, health
and queue operations

   Catalogue Admin                   Providers, products, versions, provenance, validation and publication

   Support Admin                     Support cases, messages, assignments and controlled status transitions

   Compliance / Privacy Admin        Consent, privacy, data access/deletion workflows, retention and audit
```

Read Only / Auditor	View permitted data and evidence; no mutations

The Phase 13 conversion separately reported six implemented roles: Super Admin, Business Head, Manager, Team Leader, Advisor and Viewer. These are the roles reported as built at that checkpoint. The broader role set above is the later proposed target model; do not assume they are already implemented.

Permission model

Use granular permissions in addition to roles: customers.view/update; applications.view/operate/cancel; documents.view/operate; catalogue.view/create/update/publish/archive; providers.view/manage; credit.view/configure; KYC/scan configure; notifications/support manage; consents manage; intelligence/decisions view; operations execute; retention/security configure; audit.view; admins/roles manage.

Mandatory admin safety rules

- Filament actions call existing domain services/actions; no raw DB mutation for domain state.
- Classify entities: read-only, safe CRUD, service-managed, immutable, provider-controlled, customer-owned, system-generated.
- Never manually set FynnScore, fabricate product eligibility, approve/disburse an application, fabricate provider receipt, rewrite decisions or financial observations, or mark extracted data customer-confirmed.
- Provider status must dynamically reflect reality: connected, simulated, unavailable, not configured, failed or disabled.
- Published product versions and consent versions are immutable; corrections create new versions.
- Unknown lender submission remains “needs reconciliation / do not resubmit” until a valid provider event resolves it.
- Every administrative mutation requires permission, policy/domain validation, confirmation where dangerous, audit record and refreshed state.

Audit log fields

Record admin_user_id, action, resource_type, resource_id, before/after values where safe, reason, IP, user agent and timestamp. Redact passwords, OTPs, tokens, private storage paths, raw provider payloads, raw document content and full identity numbers. Audit logs are append-only and cannot be deleted from the UI.

Other requested controls

- Customer 360 view across profile, financial data, goals, score, applications, document metadata, consents, notifications, support, intelligence, decisions and progress.
- Provider Control Center for LLM, credit bureau, OCR, KYC, lender submission/webhooks, push, email, SMS, WhatsApp, advisor and malware scanner.
- Feature flags, queue health/retry, health command integration, retention policy status, malware transfer gate, data exports with permissions/audit, global permission-aware search.
- Customer impersonation only with time-bound signed session, banner, reason and full audit; sensitive operations unavailable while impersonating.
- Admin UI responsive and premium; real query-backed metrics only; pagination, bounded queries and no N+1.

The original admin prompt requested explicit coverage for every one of modules 01–37. Each module may be a configurable screen, operational console, read-only diagnostic, immutable-history view or provider-controlled screen, but every module must have a mapped admin surface and a clear reason where changes are prohibited.

7. FynnAI strategy: Bedrock, Ollama and provider independence

A key clarification in the discussion: keep the existing FynnAI customer module as it is. The longer-term direction is a separate FynnAI Core / FynnAI Platform project that can serve FynnEdge mobile, FynnEdge website, FynnEdge LMS/RMS, MarkEdge website and future applications. Do not merge that future platform into the existing FynnAI module as part of the admin work.

Three distinct layers

Layer	Responsibility
Model	Open-weight model weights; optional future LoRA/fine-tuning. Do not train a foundation model from scratch initially.

AI Platform	Gateway, runtime abstraction, agents, controlled tools, knowledge/RAG if needed, memory, safety, evaluation and observability.
Applications	FynnEdge mobile/web, LMS/RMS, MarkEdge and future products; each connects using stable API and scoped credentials.

Proposed platform architecture

FynnAI Platform = FynnAI Gateway + Runtime + Agents + Tools + Knowledge + Memory + Safety + Models. Ollama is an inference runtime, not the product/platform. Laravel AI SDK can support Laravel-side agent/tool orchestration, but the platform boundary should remain provider-independent.

Provider routing

FynnEdge Laravel API → existing FynnAI module / gateway → Laravel AI SDK orchestration → provider interface → Bedrock or Ollama. Later vLLM may be added behind the same interface. Never let the Flutter app call Bedrock or Ollama directly; keep credentials and model access server-side.

Bedrock model discussion

The conversation identified openai.gpt-oss-20b-1:0 as the recommended initial Bedrock model to benchmark, with gpt-oss-120b as a comparison candidate rather than the default. The recommendation was based on initial cost/latency and the fact that FynnEdge's deterministic financial services provide authoritative facts while the LLM handles reasoning, tool orchestration and customer-friendly explanation. Model availability, inference mode and region must be checked in the AWS account before implementation.

Model	Use discussed
gpt-oss-20b	Initial candidate; lower-cost/lower-latency benchmark
gpt-oss-120b	Quality comparison for harder reasoning; not recommended as default at the initial stage
Amazon Nova	Alternative to benchmark, especially for multimodal/document workloads; not selected as primary

Important distinction

The Bedrock pricing table the user shared is not the same thing as a self-hosted Ollama server configuration. Bedrock is managed, token-priced inference. Ollama is software that runs a model on hardware you operate. The same model family may be available through both routes, but deployment, capacity, cost structure and operational responsibility differ.

Tools and source-of-truth boundaries

- FynnEdge remains authoritative for FinancialEngine, FynnScore, FynnMatch, FynnTwin, application status and customer data.
- Model should call allowlisted tools such as get_financial_profile, get_fynn_score, get_goals, get_fynn_match, get_loan_projection, get_application and get_financial_progress.
- Structured application data should be fetched through tools; do not dump the full customer database into the prompt.
- RAG/vector DB is optional initially. Add it only when there is a real knowledge-retrieval need.
- Memory stores consented preferences only; memory is not model training.
- No automatic training on customer financial data.

Laravel stack for platform integration

Layer	Choice discussed
Backend	Laravel 13
PHP	8.3+ (project uses PHP 8.5.4)
AI framework	Laravel AI SDK
Initial managed inference	AWS Bedrock
Self-hosted runtime	Ollama
Future high-throughput runtime	vLLM

Database	Existing project DB for FynnEdge; PostgreSQL considered for separate future AI platform
Cache / queue	Redis + Laravel Queue
Vector storage	pgvector only if/when RAG is justified
Web	Nginx + PHP-FPM
OS	Ubuntu 24.04 LTS

8. Input/output tokens and Bedrock cost example

The user shared Standard tier pricing for US East (Ohio): gpt-oss-20b at $0.07 per 1M input tokens and $0.30 per 1M output tokens; gpt-oss-120b at $0.15 per 1M input and $0.60 per 1M output. These figures are reproduced as user-provided pricing, not independently verified current AWS quotes.

Model	Input / 1M tokens	Output / 1M tokens
gpt-oss-20b	$0.07	$0.30
gpt-oss-120b	$0.15	$0.60

What counts as input?

Input tokens are the content sent to the model: system instructions, user question, conversation history, tool definitions, retrieved customer facts and tool results. Output tokens are the content generated by the model: the answer, explanation, structured response and/or tool-call text. Token counts are not identical to word counts; language, formatting and tokenizer affect the count.

Illustrative request

Input component	Illustrative tokens
System instructions	1,500
Conversation history	800
Customer question	20
Financial tool result	300
Loan projection	250
FynnScore context	150
Total input	3,020
Generated response	350 output tokens

Cost formula

Cost = (input_tokens ÷ 1,000,000 × input_rate) + (output_tokens ÷ 1,000,000 × output_rate). For 5,000 input and 1,500 output tokens, gpt-oss-20b costs $0.0008 per request at the user-provided rates; gpt-oss-120b costs $0.00165. These are examples, not fixed per-request prices.

Illustrative monthly scenario

At 10,000 customers × 20 interactions each = 200,000 requests/month, with 5,000 input and 1,500 output tokens per request: gpt-oss-20b totals 1 billion input + 300 million output tokens, costing $70 + $90 = $160/month at the supplied rates. gpt-oss-120b totals $150 + $180 = $330/month. Taxes, regional routing, caching, retries, other charges and real usage are not included.

Cost control recommendations

- Fetch only relevant customer facts for each request; avoid sending the entire profile/history by default.
- Bound conversation history and tool output; summarise older context when appropriate.
- Track model, input/output tokens, latency, tool calls and estimated cost per request.
- Measure real traffic before purchasing a GPU solely to avoid Bedrock charges.
- Compare 20B and 120B on the same FynnEdge evaluation set; select based on quality, tool-call correctness, safety, latency and cost.

9. AI server hardware and budget direction

The user asked for a budget Ollama server. The direction settled on a used RTX 3090 with 24 GB VRAM as the value-oriented GPU option, rather than immediately buying an RTX 5090 or enterprise GPU.

Component	Budget recommendation discussed
GPU	Used NVIDIA RTX 3090 24GB
CPU	AMD Ryzen 7 7700 or equivalent
RAM	64GB DDR5 (2×32GB)
SSD	1–2TB NVMe; 2TB preferred
PSU	850W quality unit minimum; 1000W Gold preferred for headroom
Motherboard	B650-class board with good VRM and physical GPU clearance
Cooling	Good tower air cooler; high-airflow case
OS	Ubuntu 24.04 LTS
Runtime	Ollama
Model	Quantized 7B–20B class; gpt-oss-20b candidate to benchmark

Why this budget direction

- GPU VRAM is the key constraint for local LLM inference; 24GB is more useful for larger quantized models than a lower-VRAM gaming GPU.
- Ryzen 7 7700 is sufficient for an initial single-GPU inference host; spend more on GPU, RAM, PSU, cooling and storage rather than an expensive gaming CPU.
- 64GB RAM is a sensible initial floor; 128GB is a later comfort upgrade if API, database, queues, model tools and retrieval services share the machine.
- Used RTX 3090 condition must be checked: VRAM stability, temperatures, fan condition, power connectors, artifacts and sustained load behaviour.
- Keep Ollama on a private network. Laravel API is the gateway; mobile clients never connect directly to Ollama.

Indicative budget

A previous conversational estimate placed a complete used-3090 build around ₹1.1–1.5 lakh, with a lower ₹70–90k development-only path based on an RTX 3060 12GB. These are rough historical planning estimates, not current vendor quotations. Actual prices depend on used-GPU condition, market availability and component selection.

Deployment topology

Early stage: one machine may host Laravel, PostgreSQL, Redis and Ollama for development/controlled testing.
Growing stage: separate the application/API server from the GPU inference server over a private network.
Higher concurrency: benchmark vLLM or multiple GPU workers only after measuring queueing, throughput and P95 latency.

Do not buy hardware before benchmarking

Test representative customer questions, tool calling, structured outputs, Hindi/English mixed queries, long context, response latency, tokens/sec, memory use and concurrent sessions.
The decision criterion is whether the system can correctly use FynnEdge tools and provide safe answers at acceptable latency—not merely whether Ollama can start a model.

10. Database, deployment and production-readiness position

Database snapshots discussed

An earlier local inspection showed DB_CONNECTION=sqlite and data stored in database/database.sqlite, with SESSION_DRIVER=database, CACHE_STORE=database and QUEUE_CONNECTION=database. At that inspected point the file was about 569 KB, with 45 tables; rows were present in users (1), customer_profiles (1), financial_profiles (1), financial_goals (3) and migrations (40). The .sqlite file was gitignored and local-only.

The later Phase 13 report states 46 tables and successful migrate:fresh verification on MySQL 8.4 and SQLite, with a preserved original SQLite checksum and a separate current database snapshot. These are different checkpoints/snapshots in the project history. Do not infer that the earlier SQLite inspection and later MySQL/SQLite verification refer to an identical current file or live production database.

Admin database principle

The Admin Panel should not have a separate customer/business database. It should operate on the same FynnEdge database and domain services used by the mobile API. A customer created/updated through an authorized admin action should be visible to the mobile API; customer changes made through the app should appear in Admin. A separate admin identity/guard is still required, as the Phase 13 report describes.

Production deployment direction

Component	Production direction discussed
Application	Laravel API + Filament admin, Nginx, PHP-FPM
Database	MySQL 8.4 is verified by Phase 13 report; PostgreSQL was recommended as an option for the separate future AI platform. Choose deliberately; do not migrate without testing.
Cache / queue	Redis recommended for production
Session	Secure production session configuration; encrypted/secure cookies
Storage	Private object storage / private disk with backups
AI	Bedrock initially or private Ollama GPU host
Network	Private API-to-inference connection; TLS at public boundaries
Backups	Database + document storage backup and restore drill
Monitoring	Health command, queue failures, provider errors and operational alerts

Production readiness reported by Module 37

Decision: ENGINEERING READY — EXTERNAL LAUNCH DEPENDENCIES PENDING. Zero P0 issues. Internal architecture, security, financial correctness, route coverage, ownership, rate limits, performance and operations were reported green. External contracts/providers and legal/product choices remain open.

Module 37 audit matrix highlights

- Architecture, security/IDOR, authentication, privacy/logging, financial correctness, FynnScore, FynnMatch, API, applications, documents/Vault, intelligence modules 31–36, concurrency, rate limits, performance, operations and Flutter: PASS as reported.
- Provider reality: 13 external provider categories absent and code defaults to none.
- Malware scanner: UnavailableMalwareScanner; transfer gate fails closed.
- Retention: periods are null by design; fynn:retention reports/refuses to invent periods.
- Production DB concurrency still needs re-verification on the actual MySQL/PostgreSQL driver; SQLite does not provide SELECT FOR UPDATE semantics.
- Low-priority cleanup: lender_tag is serialized but not rendered by a screen.

11. Decisions, constraints and items not yet verified

External integrations not connected in the reported state

Capability	Reported reality
LLM provider	None in the original 37-module FynnAI; Bedrock/Ollama discussed as next step
Credit bureau	No live bureau; driver none; fixture only
OCR/document understanding	No live provider; fixture/unavailable
KYC	No live provider; fixture only
Lender submission	No live provider; fixture only
Provider webhook source	No real provider source
Push / FCM / APNs	Not connected
Email / SMS / WhatsApp	Not connected
Advisor / CRM / live chat	Not connected

Malware scanner	Not configured; fails closed
Retention periods	Not configured; legal/product decision pending

Explicitly unverified at Phase 13

- Live success paths for credit checks, lender submission, KYC, document scanning and push.
- Flutter project’s own live API contract test.
- Real production configuration under APP_ENV=production.
- Concurrency under actual production traffic and database driver.
- Filament login driven through a real browser session.

Standing constraints

- Do not alter unrelated FYNN-ON project.
- Do not commit or deploy Phase 13 working-tree changes without user review/approval; report said 22 modified files and 13 new paths remained uncommitted.
- Do not expose model runtime or provider secrets to mobile clients.
- Do not call simulated/fixture providers live.
- Do not treat HTTP 200 from a gateway as proof of lender receipt.
- Do not retry unknown submissions until reconciled.
- Do not fabricate customer financial history, trends, approval, disbursal, score, eligibility or document verification.
- Do not invent retention periods or legal consent wording.
- Do not let admin privilege bypass domain invariants.

Separate project boundaries

The FynnAI Core/Platform is a future separate project. The FynnEdge Admin Panel may show the current FynnAI module's operational state, but must not silently absorb the future shared AI platform. The AI platform should eventually serve FynnEdge mobile/web, LMS/RMS, MarkEdge website and future apps through stable, scoped APIs.

12. Recommended next-step sequence

The following sequence is a consolidated recommendation based on the reported state; it is not a claim that these steps have already been completed.

Order	Workstream	Exit criteria
1	Review Phase 13 working tree	Review 22 modified files and 13 new paths; confirm no unintended changes; approve commit separately.
2	Admin browser validation	Drive real browser login, verify each role/navigation/action, audit trail and responsive layouts.
3	Flutter live contract test	Run api_contract_test.dart against the converted backend; retain 82/82 contract compatibility.
4	Production-like staging	Use production-like env, real database driver, Redis, secure sessions, TLS, trusted proxies and private storage.
5	Backups and restore	Automated DB/storage backups; complete a restore drill; verify RPO/RTO expectations.
6	OTP provider	Choose/configure real OTP provider; test deliverability, rate limits, abuse controls and failure paths.
7	Admin control plane coverage	Build capability matrix for all 37 modules; granular permissions; audit all mutations; keep immutable/provider-controlled records protected.
8	FynnAI evaluation set	Create 200–500 representative questions; test tool calling, accuracy, financial safety, Hindi/English, prompt injection, latency and cost.
9	Bedrock pilot	Benchmark gpt-oss-20b first and gpt-oss-120b as challenger using user-provided price sheet; track tokens and cost.

10	Ollama pilot	On a test host or used RTX 3090 system, benchmark the same model/workload; compare cost, throughput and quality.
11	Provider/legal readiness	Obtain real contracts, credentials, retention periods, data-location/training/deletion terms and webhook specifications before enabling external data flows.
12	Launch decision	Only after unresolved production blockers have named owners, evidence and explicit approval.

Suggested FynnAI evaluation categories

- Loan affordability and EMI explanation using tool results.
- FynnScore explanation without claiming it is CIBIL or an approval score.
- FynnMatch criteria and unavailable-rule explanations.
- Goal planning and financial shortfall safety.
- Application status explanations that distinguish FynnEdge state from lender/provider state.
- Questions with missing/inconsistent data; model should ask for clarification rather than invent.
- Hindi, English and mixed-language queries.
- Prompt injection and requests to reveal other customers' data.
- Requests for guaranteed approval, fabricated provider status or prohibited identity details.
- Tool-call accuracy, structured-output validity, latency, token usage and cost.

Recommended operational principle

Keep the deterministic FynnEdge domain layer authoritative. Use the LLM for language, reasoning over returned facts and controlled tool orchestration. Use Admin for visibility, governance and approved operations. Keep provider status honest. Measure before scaling infrastructure.

Appendix A. Core implementation prompt principles

The long Admin Control Plane prompt provided in the conversation asked Claude Code to inspect the full backend before coding, document existing models/migrations/services/policies/providers/routes/enums/auth/audit/health/queue/configuration, then build an ADMIN_ARCHITECTURE.md and ADMIN_CAPABILITY_MATRIX.md before generating Filament resources.

Required capability matrix columns

Module; Entity; View; Create; Edit; Delete; Publish; Retry; Reconcile; Customer Visible; Provider Visible; Immutable; Required Permission; Domain Service; Audit Required.

Required implementation sequence

- Admin Phase A: authentication, roles, permissions, audit, navigation, dashboard, health.
- Admin Phase B: customers, financial profile, goals, applications, documents, consents, support, notifications.
- Admin Phase C: FynnScore, FynnMatch, FynnTwin, intelligence, coach, plan, progress, review, decision history.
- Admin Phase D: providers, catalogue, versions, credit, OCR, KYC, lender submission, provider events.
- Admin Phase E: queues, retention, malware, feature flags, settings, system health and exports.
- Admin Phase F: final security/audit audit.

Required final acceptance

- 37/37 modules mapped to an admin surface.
- Admin authentication is separate from customer authentication.
- Every mutation uses the correct service and produces audit evidence.
- Provider-controlled outcomes and immutable historical records remain protected.
- Customer tokens cannot reach admin; admin sessions do not become customer API tokens.
- No secrets, OTPs, tokens, storage paths, raw document content or raw provider payloads leak.
- Existing 37-module tests remain green; route/security/performance/UI tests pass.
- Final ADMIN_IMPLEMENTATION_REPORT.md includes coverage, permissions, services, audit, security, provider reality, tests and remaining debt.

Appendix B. Key terminology

Term	Meaning in this project
FynnScore	Internal financial-health indicator; not a credit bureau score or approval probability.
FynnMatch	Deterministic product matching using known/published criteria.

```
   FynnTrust                        Product transparency/evaluation presentation.

   FynnTwin                         Non-persistent simulation of possible financial scenarios.

   FinancialEngine                  Authoritative deterministic calculations for financial position and
projections.

   ApplicationStatus                What the customer did within FynnEdge.

   SubmissionOutcome                What FynnEdge knows about outbound provider submission.

   ProviderEvent                    Authenticated inbound statement from a provider.

   CustomerStatus                   Customer-readable presentation derived from real application/submission
facts.

   Fixture provider                 Test-only simulated adapter; never a live integration.

   Unknown submission               Outcome is not confirmed; reconcile, do not resubmit.

   FynnAI Core                      Future separate shared AI platform; not the same as the original FynnAI
customer module.

   Input tokens                     Instructions, question, history, tool definitions and retrieved context
sent to the model.

   Output tokens                    Text/structured response generated by the model.

   Ollama                           Self-hosted inference runtime, not the entire AI platform.

 Record limitations
 This PDF is a structured reconstruction from the conversation context, stored project
 summaries and the Phase 13 report text supplied in the conversation. It does not reproduce
 every historical chat message verbatim. Where an exact prompt/report was not available in
 full, the record preserves its stated requirements and decisions at summary level.
 User-provided pricing and hardware estimates are labelled as such; no claim is made that
 prices are current vendor quotes.

                                End of FynnEdge Complete Project Record
```

---

5. Source B: Flutter app README

`fynnedge/README.md`, verbatim. Written during the build; file dated 8 Sep 2026.

````
# FynnEdge

**FynnEdge Advisory — Simplifying Loan, Amplifying Trust.**

A standalone Flutter application. It shares no code, database, API, auth or
models with `fynn_on` or any other existing system.

---

## Running it

```bash
flutter run                      # mock data, no backend needed
flutter run -d linux             # desktop window is phone-sized for review
```

The app boots straight into the customer journey with realistic development
data. No backend, OTP provider, AI key or lender integration is required.

---

## Swapping mock services for your real API

This is a one-line change, and no UI or controller code moves.

Every service has two implementations behind one interface — for example
`HomeService` is implemented by both `MockHomeService` and `ApiHomeService`.
The choice is made in exactly one place, [`lib/app/providers.dart`](lib/app/providers.dart),
driven by a compile-time flag:

```dart
final homeServiceProvider = Provider<HomeService>(
  (ref) => ApiConfig.useMock
      ? const MockHomeService()
      : ApiHomeService(ref.watch(apiClientProvider)),
);
```

To point the app at a real backend:

```bash
flutter run \
  --dart-define=FYNN_USE_MOCK=false \
  --dart-define=FYNN_API_BASE=https://api.fynnedge.com
```

Screens, controllers and repositories are untouched by this — they only ever
see the interface.

### The data flow

```
Screen  →  Riverpod provider  →  Repository  →  Service  →  API or mock
```

- **Screens** render. They never call a service or build a URL.
- **Repositories** compose and cache. `LoanRepository` is the interesting one:
  it prices raw products into offers using the local finance engine, so EMI
  figures are identical whether the catalogue came from mock data or the wire.
- **Services** are pure transport. One interface, two implementations.

### Where the fake data lives

All of it is in [`lib/data/mock/mock_data.dart`](lib/data/mock/mock_data.dart)
and nowhere else. Lender names are fictional on purpose so screenshots can
never be mistaken for live market pricing.
````

```
---

## Financial calculations are never mocked

[`FinanceService`](lib/data/services/finance_service.dart) is real arithmetic
that runs on-device and always will. EMI, amortisation, affordability,
prepayment, balance transfer and emergency-fund maths do not depend on the
network, and a backend swap cannot change what a number means.

It is covered directly by [`test/widget_test.dart`](test/widget_test.dart) —
the standard reducing-balance formula, verified against known values.

---

## The financial review

Reached from Profile, because the customer asks for it. It gathers and
computes nothing: the position, the score, the goals, the observed changes
and the next step all come from **one** `/intelligence` read, so no two
sections can describe the customer at two different instants. Recorded
decisions come from their own request — an immutable fact with its own
timestamp cannot contradict a figure.

Sections with nothing to say are absent rather than filled. A customer who
has entered nothing sees what cannot be worked out and one step that leads
somewhere — no ₹0 surplus, no zero-month buffer, no invented goal.

There is no review score, no grade and no schedule. Nothing records that a
review happened, so there is no honest "last reviewed" date and the screen
does not invent one.

---

## Progress and the story

The story screen has two halves and the difference matters. **Progress** is
what FynnEdge observed *changing* — a figure it wrote down twice with
different values. The **story** beneath it is what *happened*: goals set,
applications started, a score becoming available.

The screen derives none of it. Both sentences and both figures come from the
server, so a history cannot print a number differently from the screen it
came off. There is no chart, deliberately: a line needs points between the
ones that exist, and inventing them turns a record of what happened into a
picture of what might have.

The empty state says which kind of empty it is. "Nothing to compare yet" and
"no changes recorded" are different facts, and a customer who has just
arrived deserves the first rather than being told their money has not moved.

Nothing is coloured by direction — recorded outgo going up is not bad news
and going down is not good news — and each row announces itself as a
sentence with the direction in words, because an arrow is legible on a
screen and silent when read aloud.

---

## The financial plan

The Goals screen is the plan. It renders three answers to one read of one
customer: the position the server worked out, the goals the customer
created, and the next step from the same action ladder Home reads. It
computes none of them — which is why the card on Home and the plan here
cannot suggest different things.

Two rules are worth knowing before editing it.

**The borrowing shortcut is withheld from a customer whose recorded outgo
exceeds their recorded income**, and also while the position has not loaded,
because FynnEdge does not then know the surplus is safe. `plan_test.dart`
checks both the absence and — on the same card, with a surplus — its
presence, because a test that only ever asserts an absence passes just as
well when the thing it guards was never reachable.
```

Nothing on the screen predicts. The monthly figure is what the
arithmetic gives at the target and date the customer chose, and it changes
the moment they change either. It used to read "Set aside ₹9,474 a month for
19 months to get there", and Home used to print "/mo to stay on track" —
both claims about a future FynnEdge cannot see. `product_language_test.dart`
now scans for the phrasings on both sides of the product.

Tests

```bash
flutter test                      # everything; the API suite skips if no backend
flutter test --update-goldens     # refresh the screenshots in test/goldens
```

Suite	What it guards
`finance_test.dart`	The EMI engine, cross-checked against a month-by-month simulation that must close the balance to zero. Independent of the closed-form formula.
`model_parsing_test.dart`	Every model against the awkward JSON a real backend emits — nulls, absent keys, `[]` where an object was expected, numbers as strings, unparseable dates.
`screens_test.dart`	Renders all 29 screens and writes a PNG per screen. A screen that overflows, throws or reads a missing provider fails here.
`responsive_test.dart`	Every screen at 360x640, 390x844 and 430x932, plus small-phone goldens for the dense screens.
`keyboard_test.dart`	Input screens with a raised keyboard, including that the CTA can still be reached.
`navigation_flow_test.dart`	Drives the real router through the full 25-screen journey, signup included.
`routes_test.dart`	Every constant in `Routes` resolves to a real screen, so a renamed path cannot leave a dead link.
`api_contract_test.dart`	Parses live Laravel responses through the real models, error paths included.
`review_test.dart`	The financial review: which sections a customer earns, and what it must never say however complete the rest looks.
`progress_test.dart`	What the story screen may show about change, and what it must never say. Both halves of every guard: the state that suppresses something and the state that shows it.
`plan_test.dart`	The financial plan: what it may show in each customer state, and what it must never say. Every guard is checked twice — once for the state that suppresses something, once for the state that shows it.

The API suite needs the backend:

```bash
cd ../fynnedge-api && php artisan serve --port=8123
flutter test --dart-define=FYNN_API_BASE=http://127.0.0.1:8123
```

Determinism

Anything the customer sees that depends on the current time reads
`AppClock.now()`, never `DateTime.now()`. Tests freeze it, so greetings,
relative timestamps and seeded dates do not drift and goldens stay stable.

Structure

```
lib/
  app/            theme, router, routes, Riverpod wiring (providers.dart = the swap point)
  core/           clock, formatters and the shared widget kit
  data/
    models/       plain classes with fromJson/toJson
    services/     transport: one interface, a Mock and an Api implementation
    repositories/ what controllers talk to
    mock/         every fake value in the app, in one file
  features/       one folder per area, screens + their controllers
```

State management is **Riverpod only** — no Bloc, Provider or GetX mixed in.

What is deliberately not real yet

> Placeholders with clean integration points, waiting on your providers:
>
> | Area | Today | Swap in |
> |---|---|---|
> | OTP | any 6 digits except `000000` | your SMS provider, in `AuthService` |
> | Lender offers | 6 fictional products | real catalogue, in `CatalogService` |
> | FynnAI | rule-based, reads real financials | a model provider, in `AiService` |
> | Document OCR | staged mock extraction | real pipeline, in `DocumentService` |
> | Storage | local | private cloud, in `DocumentService` |
> | Credit data | not called | bureau integration |
>
> Applications are read-only — submission is stubbed until lender integrations exist. Two FynnLab tools (Debt Calculator, Future Simulator) are listed but marked "soon" rather than faked.

---

6. Source C: API README

`fynnedge-api/README.md`, verbatim (885 lines). Written during the build; last committed 10 Sep 2026 (`3b335a8`).

````
# fynnedge-api

Laravel backend for the **FynnEdge** Flutter app.

Standalone. It shares no code, database, authentication or infrastructure
with fynn_on, FYNN-ON, the LMS, the CRM or any existing loan system, and it
must not be connected to them.

Two front doors onto one application:

```text
Flutter app ──► /api/v1/*  ──► Sanctum bearer token ──► users
                     │
               shared services, models, policies, database
                     │
FynnEdge staff ──► /admin  ──► session (admin guard)  ──► admin_users
```

The two authentication systems are separate by construction, not by a
check somebody has to remember: a customer token names a `User`, the admin
guard's provider only builds `AdminUser`, so there is nothing at `/admin`
for a customer token to be.

## Requirements

| | |
|---|---|
| PHP | 8.3+ (developed and tested on 8.5) |
| Composer | 2.x |
| Database | MySQL 8.x in production; SQLite for the test suite |
| Node | only to build the admin panel's assets — the API needs none |
| Extensions | `intl` (Filament), plus the Laravel defaults |

## Installation

```bash
composer install
cp .env.example .env
php artisan key:generate

# Create the schema. `migrate` is additive and safe to re-run;
# `migrate:fresh` drops everything and is not for a populated database.
php artisan migrate

# Baseline development data — one customer, matching the app's own mocks.
php artisan db:seed
```

Then, for development only:

```bash
php artisan db:seed --class=AdminUserSeeder      # a Super Admin for /admin
php artisan db:seed --class=SampleCatalogueSeeder # the six sample products
php artisan db:seed --class=DemoFixtureSeeder     # four invented customers
```

All three refuse to run when `APP_ENV=production`. The first behaves
differently there — see **The admin panel** below.

## Running

```bash
php artisan migrate
php artisan serve --port=8123
````
````

````
# Document reading runs on the queue. With QUEUE_CONNECTION=sync (the
# default) it runs inline and no worker is needed; with `database`, run:
php artisan queue:work
```

All routes are under `/api/v1` (set in `bootstrap/app.php`).

```bash
php artisan route:list --path=api
```

## The API

82 endpoints, all under `/api/v1`. Three are public — the two OTP endpoints
and the provider ingress — and every other one requires a Sanctum bearer
token.

**Signing in.** There are no passwords. A customer is a mobile number that
proves itself with a one-time code:

```bash
POST /api/v1/auth/send-otp     {"mobile":"9876543210"}
  → {"data":{"reference":"…","expires_in":30,"dev_hint":"123456"}}

POST /api/v1/auth/verify-otp   {"mobile":"…","reference":"…","code":"…","device_name":"Pixel 8"}
  → {"data":{"token":"…","is_new_user":false,"expires_at":"…","profile":{…}}}
```

`dev_hint` is the issued code, returned only when `FYNNEDGE_OTP_DEV_HINT` is
explicitly on **and** the app is not in production. Both conditions have to
hold; see **Environment**.

**Every other request** carries the token:

```http
Authorization: Bearer <token>
Accept: application/json
Content-Type: application/json
```

Tokens last 30 days (`SANCTUM_TOKEN_EXPIRATION`, in minutes), are stored only
as a SHA-256 hash, and are revoked per device — signing out on a phone leaves
a tablet signed in.

**Responses** are always enveloped:

```jsonc
{"data": {…}}                 // or  {"data": [...]}
{"data": [...], "meta": {…}}  // where a list paginates
{"message": "…"}                          // errors
{"message": "…", "errors": {"field": ["…"]}}  // validation
```

**Status codes** the app should expect:

| | |
|---|---|
| 200 / 201 / 204 | read · created · deleted |
| 401 | no token, or it is expired or revoked |
| 404 | not found **or not yours** — deliberately indistinguishable |
| 409 | idempotency conflict |
| 422 | validation, or the domain refusing |
| 429 | rate limited; honour `Retry-After` |
| 501 | the capability exists but no provider is connected |

**Ownership.** Every customer-scoped route resolves through the token
holder's own relation, so another customer's id answers **404 rather than
403** — an attacker cannot tell "not yours" from "does not exist".

**Uploads** go to `POST /api/v1/documents` as `multipart/form-data` with
`file`, `type` and `name`. `GET /api/v1/documents/limits` states what is
accepted (10 MB; PDF, JPEG, PNG, HEIC, WebP) so the app can refuse before a
round trip. Content is sniffed, not trusted — a renamed executable is
rejected. Files stream back through `GET /api/v1/documents/{id}/download`,
````
````

````
authenticated on every request; there is no public or signed URL.

**Rate limits** are per capability, not global: OTP sends 3/min per number,
credit checks and KYC 5/hour, lender submissions 3/hour, vault writes 20/min,
authenticated traffic 300/min. The full list is in
`AppServiceProvider::configureRateLimiting()`, each with the reason for its
figure.

The complete inventory:

```bash
php artisan route:list --path=api
```

## The admin panel

Filament 5 at `/admin`, for FynnEdge staff. It is not the API and the app
never touches it.

```bash
php artisan db:seed --class=AdminUserSeeder
# Development Super Admin: admin@fynnedge.test
# Password: <generated, printed once>
```

The password is never written down here. Set `FYNNEDGE_ADMIN_PASSWORD` to
choose one, or leave it unset and the seeder generates a password and prints
it to the console of whoever ran it — after which only the hash exists. An
administrator that already exists keeps the password it has; the seeder will
not silently reset it.

In production the same seeder creates nothing unless `FYNNEDGE_ADMIN_EMAIL`
and `FYNNEDGE_ADMIN_PASSWORD` (12 characters minimum) are both set, and it
refuses rather than inventing a default — a well-known credential on a panel
that reads every customer's finances is the whole breach.

**Six roles**, and what each may reach:

| Role | Staff accounts | Customer money | Identity & credit | Catalogue | Support |
|---|---|---|---|---|---|
| Super Admin | ✓ | ✓ | ✓ | ✓ | ✓ |
| Business Head | | ✓ | ✓ | ✓ | |
| Manager | | ✓ | ✓ | | ✓ |
| Team Leader | | | | | ✓ |
| Advisor | | | | | ✓ |
| Viewer | | | | | |

Every role can see customers and consents; staff cannot answer for the
product otherwise. `Advisor` is the application's own word — support messages
have carried `author_type = advisor` since Module 30.

**Almost everything is read-only.** Customer records are written by the
domain services, which also record *why*; a panel that could edit the row
directly would be a second writer with no such account of itself. The
exceptions are staff accounts, the catalogue, and replying to a support case
— and that reply goes through `SupportCaseService::addAdvisorMessage()`, the
same service an advisor API would call.

**Deactivate, never delete.** `is_active = false` ends a session already in
progress, checked on every request rather than only at login. Nothing in the
panel deletes a customer, an application or a member of staff, because those
records are the evidence behind decisions.

Sensitive fields are absent rather than masked — a masked figure still
confirms one exists. Document storage keys, bureau references, KYC identity
fields, file digests and every password or token hash are withheld from all
roles, and that is asserted against the rendered HTML in
`SensitiveExposureTest`.

## Database

**Migrations are the schema. Seeders are the data.** The application does not
depend on any pre-existing database file to rebuild itself:
````

```bash
php artisan migrate:fresh   # 41 migrations, from empty
php artisan db:seed         # baseline development data
```

41 migrations produce 46 tables, 502 columns, 41 foreign keys and 130
indexes. `php artisan migrate:fresh` is verified to work from nothing on both
MySQL and SQLite.

Development data comes from four seeders, each opt-in except the first:

Seeder	Writes	In production
`DemoCustomerSeeder`	the one customer the app's mocks mirror	refuses
`AdminUserSeeder`	a Super Admin	requires env vars, or refuses
`SampleCatalogueSeeder`	six fictional products	refuses
`DemoFixtureSeeder`	four invented customers with applications, documents, cases	refuses

Demo customers use the reserved `90000001xx` block and every record is
labelled — names begin "Demo", references begin `DEMO-`. Remove them with
`php artisan demo:purge`.

What is connected, and what is not

FynnEdge is deliberately explicit about which capabilities are real. Each of
these defaults to `none` or to sample data, and each says so to the customer
rather than simulating the real thing.

Capability	Setting	Default	Status
Product catalogue	`FYNNEDGE_CATALOGUE_DRIVER`	`sample`	Six invented products, labelled as sample data
Provider ingestion	`FYNNEDGE_PROVIDER_DRIVER`	`fixture`	**No live provider connected**
Credit bureau	`FYNNEDGE_CREDIT_DRIVER`	`none`	**No live bureau connected**
FynnScan (document understanding)	`FYNNEDGE_SCAN_DRIVER`	`none`	**No live OCR/document provider connected**
KYC (identity verification)	`FYNNEDGE_KYC_DRIVER`	`none`	**No live KYC provider connected**
Lender submission	`FYNNEDGE_LENDER_DRIVER`	`none`	**No live submission provider connected** — applications are recorded and sent to nobody
Provider events	`FYNNEDGE_PROVIDER_EVENTS_DRIVER`	`none`	**No live provider event source connected** — the ingress allowlist is empty
FynnAI	`FYNN_AI_PROVIDER`	`deterministic`	Answers from the customer's own saved figures; not a language model
Push notifications	`FYNNEDGE_PUSH_DRIVER`	`none`	**No live push provider connected** — no FCM, no APNs, no credentials, no device tokens
Advisor / support CRM	—	—	**Not connected.** No advisor system, ticketing system, CRM or chat provider. Support cases live in FynnEdge's own tables and are read by whoever looks at them
Language model / predictive model	—	—	**Not connected, and not required.** The intelligence layer is deterministic domain rules; FynnAI reads it rather than producing it
Analytics platform	—	—	**Not connected.** No warehouse, event stream or feature store. Financial observations are written at meaningful change only
Email / SMS / WhatsApp notifications	—	—	**Not implemented.** No channel exists and none is stubbed
Marketing communication	—	—	**Not implemented.** Nothing composes, schedules or sends one
OTP delivery	`FYNNEDGE_OTP_PROVIDER`	development	Returns a `dev_hint` instead of sending an SMS

`.env.example` documents every variable, including what has to be settled
before any of these can go live.

Architecture

Authentication is Sanctum. Everything except `/auth/send-otp` and
`/auth/verify-otp` sits behind `auth:sanctum`. Ownership always resolves
through `$request->user()->relation()->find($id)` — never route-model
binding — so another customer's id returns the same 404 as one that does not
exist.

Money is integer paise throughout (`App\Domain\Money`). No float touches
a rupee figure.

Mirrored engines. The financial, score, match and twin engines exist in
both PHP and Dart and are proven identical by committed parity fixtures:

```bash
```

```
php artisan fynn:parity-fixture   # regenerate after changing an engine
```

Catalogue. `Provider → CatalogueProduct → ProductVersion → LoanProduct` behind `ProductRepository`. Published versions are immutable; applications snapshot their pricing, so republishing a product changes only what new customers see. Administered by `php artisan fynn:catalogue` — there are no admin HTTP endpoints, deliberately.

Credit. `Consent → CreditBureauAdapter → normalizer → CreditProfile → credit criterion → FynnMatch`. FynnEdge stores a score, its model, its scale and when it was obtained — never the report. A bureau score is not a FynnScore and nothing converts one into the other.

FynnScan. `Vault → consent → transfer gate → queue → worker → DocumentUnderstandingProvider → normalizer → canonical extraction → customer review → confirmation → financial profile`.

Reading happens on the queue, not in the customer's request. The worker re-checks everything the request checked — the document still exists, its bytes are unchanged, and consent still stands — because by the time it runs any of those may have changed. Set `QUEUE_CONNECTION=database` and run `php artisan queue:work` in production; the `sync` default runs it inline. Extraction is evidence, not the customer's data: nothing read from a document reaches their profile until they confirm that specific field. FynnScan reads income documents only.

Lender submission. `Application → consent → LenderApplicationMapper → LenderSubmissionProvider → canonical outcome`. A FynnEdge application is not a lender application: creating `FE-2026-000001` means the customer completed an application here, and only a provider's own acknowledgement can mean an external party has it. The two live on separate axes — `ApplicationStatus` for what the customer did, `SubmissionOutcome` for what a provider did — and `provider_received` is the single field that means an external party holds it. A 200 from this API never sets it.

Every field that leaves is listed in `App\Domain\Lender\SubmissionFieldMap` with its purpose, and everything refused is listed there with the reason: FynnScore, FynnTrust, Twin scenarios, credit data, scan extractions, KYC results, documents, savings, goals and AI conversations do not go.

Provider events. `Ingress → provider authentication → normalizer → canonical event → idempotency → reconciliation → submission and application state → timeline`. Deliberately outside `auth:sanctum`: a customer's token authenticates nothing there, and a callback carries no customer session.

Three states stay separate. `ApplicationStatus` is what happened inside FynnEdge; `SubmissionOutcome` is what FynnEdge knows about the delivery; a provider event is what a provider said. Only a valid provider event can write `under_review`, `approved`, `declined` or `disbursed`, and only a provider event can resolve a submission whose outcome is `unknown`.

An event that arrives before FynnEdge has stored the reference it quotes is recorded as failed rather than lost; `php artisan fynn:provider-events:retry` re-attempts those through the same transition policy, updating the original row rather than creating a second. It is deliberately not scheduled — it mutates customer application state, and running it unattended is a deployment decision to take on purpose.

Events are idempotent on a provider event id, or on a deterministic fingerprint where a provider issues none. Ordering is by event meaning — rank must strictly increase — so a late event cannot undo a later one and two contradictory decisions do not flip a customer's outcome on delivery order. Nothing of a callback's body is stored beyond a hash.

KYC. `Vault → consent → KycVerificationProvider → normalizer → canonical verification → customer review → optional profile update`. A separate capability from FynnScan on the same vault, with its own consent, because reading a document and checking an identity are different questions.

Two findings are carried apart and never merged: what the service's own checks said about the document, and whether the identity on it matched the customer's profile. A verified result is not approval and changes nothing on a profile by itself. Only `identity` documents are checked; the only field

compared is name; and only a masked tail of a document number is stored —
never a full PAN, never an Aadhaar number, never a date of birth or address.
`App\Domain\Kyc\IdentityFieldPurpose` records why each field is kept or
refused, and `App\Domain\Kyc\AadhaarBoundary` records why FynnEdge does not
touch Aadhaar authentication.

Notifications

Two rules shape the whole layer.

An internal event is not a customer notification. FynnEdge records far
more than a customer needs to read — every submission attempt, every
provider event including the ones it ignores, every profile edit. A
notification for each would be a log, and a log is not communication. So
`NotificationPolicy` is the only thing that decides, it returns a decision
or `null`, and `null` is the ordinary answer.

Creating a notification is not delivering a push. They are separate
tables for that reason. A push that fails, a provider that is not
configured, a customer with no device and a customer who turned push off
all end the same way for the notification: it is in the notification
centre, unchanged, waiting to be read.

```
Existing domain event  →  NotificationPolicy  →  notifications row
                                              →  DeliverNotification job
                                              →  NotificationDeliveryProvider
```

`NotificationDecision` is the canonical object every producer must build,
and it takes two things away from callers entirely: the **category** and
whether the notification is **transactional** are both derived from the
type. A caller cannot file a document problem under goals, and cannot
classify "action required" as optional. It also refuses a link the router
cannot open, and copy containing anything shaped like a PAN, a long number,
a storage key, a digest or a token.

Idempotency is a unique index on `(user_id, idempotency_key)`, reached
through `insertOrIgnore`. Not an `if exists` check — provider redelivery,
queue retries and a double-tapped button all aim at the window between the
read and the write. Eight forked processes recording one event produce one
row, and exactly one of them believes it created it, which is what stops a
customer being pushed eight times. Provider-event idempotency is separate
and stays in Module 28.

Ordering is inherited, not reimplemented. Provider-event notifications
are created inside `SubmissionReconciliationService`'s transaction on the
applied branch only, so an event Module 28 ignored as stale cannot announce
"under review" after a disbursal.

Attribution. A provider's decision is the provider's. FynnEdge never
approves, declines or disburses anything, and the attribution is in the
title so it survives a truncated push.

Read state is server-side and idempotent: the first `read_at` stands, so
reading twice does not move when they first saw it. `read-all` is safe to
repeat and returns zero the second time.

Preferences change how FynnEdge reaches out, never what it records.
Turning push off does not empty the notification centre. Only categories in
which *every* type is non-transactional can be muted — derived from the
types themselves, so adding a transactional type closes a category
automatically. Today that is `financial_health` and `goal`; applications
and documents cannot be muted at all. Marketing consent is a separate thing
in a separate place and never gates service communication.

Deep links are one contract, shared by `App\Domain\Notifications\DeepLink`
and Flutter's `DeepLinks`: `fynnedge://applications/{id}`,
`fynnedge://vault/{id}`, `fynnedge://fynn-score`, `fynnedge://goals`,
`fynnedge://support`. There is deliberately no per-goal link, because there
is no per-goal screen. A link the app does not recognise opens nothing.

Retention is unset. Notification history is kept. No period has been
decided by product or legal, and `fynn:retention` does not delete

```
notifications — inventing a period to make the module feel finished would
be deleting a customer's own record of what they were told.

### Decision history and explainability

**What is recorded, and what deliberately is not.** Of the decision contexts
a customer could be said to explore, only **two** reach the server as an
action: a FynnTwin simulation and a product comparison. Both are recorded.

Nothing else is. `product_detail_review` and `fynntrust_review` would be
records of a GET — page-view tracking however they are labelled — and
FynnTrust and FynnMirror have **no API endpoint at all**, so recording them
would mean inventing a tracking endpoint to build a history of what somebody
looked at. Opening a product is not wanting it.

**Applications are read, not copied.** `loan_applications` has held an
immutable snapshot of exactly what the customer was shown since Module 9 —
amount, tenure, rate, EMI, fees and the catalogue version behind them. A
second copy could disagree with the first, and the customer would be shown
whichever the code happened to read. There is also **no `application_id` on
a decision**: FynnEdge cannot know a simulation led to an application
without inferring it from "same product, later", and a column that could
only be filled by a guess will be.

**Historical integrity.** An explanation is built from figures recorded at
the time, never recomputed. A decision from six months ago explains itself
with the rate it was actually shown — verified live by dropping a customer's
income from ₹1,50,000 to ₹40,000 and re-reading the decision, which still
says ₹1,50,000, noted "as it was at the time".

**Provenance on every figure.** `customer_provided`, `calculated`,
`catalogue_published`, `provider_derived`, `simulated`,
`historical_snapshot`. There is no `estimated` and no `predicted`: FynnEdge
holds a figure, was given one, or worked one out.

**What was not assessed is said out loud.** An affordability explanation
carries "Not assessed: whether a lender would approve this" rather than
leaving a gap — a gap where approval would be is a gap a customer reads as
a pass.

**`DecisionSnapshot` is the canonical boundary**, with an allowlist of
figures and a value check on top: an allowed key carrying a PAN is the same
disclosure as a key called `pan`. A comparison records no winner, because
there was none.

**Deduplication:** the type, figures, product and day. A slider that lands
back on the same figures is one exploration; anything different is its own.
Proved with eight forked processes.

**Retention is unset.** A decision history is a customer's own record of
what they were shown. No period has been decided and nothing deletes it.

### Financial intelligence

**The finding this module is built on:** FynnEdge kept **no financial
history at all**. `financial_profiles` is a single mutable row, so editing
an income destroyed the previous one irrecoverably, and no FynnScore had
ever been remembered.

That makes a whole class of claim impossible rather than merely
unimplemented — "your income increased 12%", "your expenses are falling",
"your score improved by 6", "this goal has stalled". So FynnEdge starts
observing, and **starts empty**: no row in `financial_observations` is
reconstructed from today's profile, so no existing customer acquires a
history they did not have. Until a customer has been observed twice, the
honest answer to "what has changed?" is that FynnEdge does not know, and
`history.has_comparable_history` says so explicitly rather than leaving a
client to infer it from an empty list.

Observations are written at **meaningful change only** — a profile edit, a
goal change, an application milestone. Never a page view: an app open is not
a financial event, and a table that grew on refreshes would be a record of
somebody's attention rather than their money.
```

```
Domain event  ≠  Financial Moment  ≠  Notification
```

A **moment** is one of the handful of events a customer would recognise as part of their own story. Moments are **historical facts and are never rewritten**: a moment saying a score rose from 64 to 71 keeps saying that after the score falls to 55. That is why the sentence is written at the time and stored — deriving it on read would mean deriving it from today's state, which is exactly the rewriting the table exists to prevent. Duplicates are prevented by a unique index on `(user_id, idempotency_key)`, proved with eight forked processes.

Insights are the opposite: derived on every read and never stored, so a recommendation cannot outlive its condition. A score change is deliberately **not** an insight — it is something that happened, so it lives in the timeline.

The action plan is that ladder's full output, capped at three. The single next action is its first item, so Home's card and the plan beneath it come from one engine and cannot suggest different things. It is allowed to be short or empty: a customer whose finances are in order is told so rather than handed an invented task.

Two rules exist to keep it a planner rather than a funnel. A customer whose recorded outgoings exceed their income is offered a review of their own figures and is **never** pointed at borrowing — that one was found by probing, because the three-item cap had hidden it from the test written for it. And a strong FynnScore produces no improvement action at all.

Next best action is a single answer from a deterministic ladder: blocked → missing input → active decision → goal deadline → getting started → nothing needed. The first rung exists because of a failure it prevents: Home used to answer *"Set your first goal"* to a customer who had been told not to resubmit an application FynnEdge could not confirm. Home now consumes these rules rather than running its own, so the two cannot disagree.

Thresholds are existing product definitions — the 45% affordability reference and the six-month emergency target, both FinancialEngine's. Module 31 introduces one of its own, on `ScoreChangePolicy`, with the reasoning beside it.

Explainability. Every insight, next action and moment records the rule that produced it and the values it fired on. None of that reaches the customer — it describes the system — and a test fails if anything ships without it.

Retention is unset. Moments and observations are a customer's own record of their financial life. No period has been decided, nothing deletes them, and `fynn:retention` does not touch them.

The financial review

The customer asking to be shown where they stand. **There is no review table, no `/financial-review` endpoint and no review engine** — the review is the `/intelligence` read, and every section of it already existed somewhere: the position is `FinancialPosition`, the score is `FynnScoreEngine`'s, the goals are the Goals module's own metrics, the changes are Module 35's observations and the next step is Module 33's ladder.

`/intelligence` therefore carries two things it did not before — `score` and `goals` — both from the state the request already built, at no extra query. That is what makes the review one coherent moment: a goal edited between two requests could otherwise let the goal section contradict the next step beside it.

Asking for a review changes nothing. No observation, no moment, no decision, no notification. Because nothing is recorded, FynnEdge has no honest way to say when the last review was or when the next is due, and the screen says so by not saying it: no schedule, no reminder, no streak.

What it deliberately does not have is a verdict. No review score, no completeness percentage, no health grade. FynnScore already has bands, and a

```
second opinion about somebody's finances is exactly what a review must not
be. `test/product_language_test.dart` scans both codebases for the grading
and scheduling vocabulary.

**One score, one set of goal figures.** `FynnScore::toSummaryArray()` is the
only place a band is turned into a label and a sentence, so the review and
the FynnScore screen cannot phrase one differently. The goals come from the
metrics `FinancialEngine` already produced for the snapshot, bounded at ten
with the true count beside them.

### Financial progress

What FynnEdge has actually observed changing, and nothing else. Every item
on the progress feed comes from a pair of consecutive rows in
`financial_observations` — **today's figures are never one end of a
comparison**, because they are what is true now rather than something
FynnEdge wrote down, and pairing the two would invent the second half of
every entry.

`FinancialProgress` reads the observations and returns the metrics that
moved. It is silent by default, and each silence has a reason:

| Situation | Why nothing is said |
|---|---|
| One observation | A baseline, not a trend |
| Identical values | Not a change, however much else moved |
| A score arriving | An arrival — Module 31 records it as a moment |
| A score across model versions | A different measurement; subtracting is meaningless |
| A move under `ScoreChangePolicy::POINTS` | The threshold the product already uses |
| Either end unmeasured | Zero and absent are different facts |
| A buffer with no outgo | No denominator, so no figure that can have changed |

**A score change explains itself from what was true at the time.** The model
reads three ratios; both observations stored them; so each component is
reported as the value it had when each score was calculated, scored by
FynnScore's own published curves. Nothing is recomputed from the current
profile, which is what keeps a historical explanation still true tomorrow.
It is never a cause: several figures move between two observations and the
model averages three of them.

**Deduplication is enforced by the database.** `financial_observations`
carries a `state_digest` and a `follows_id`, unique together per customer,
reached through `insertOrIgnore`. An observation identical to the one it
follows is not a new observation; a state a customer returns to later is,
because the row before it is different. The recorder also compares before
writing — that comparison handles the sequential case and the index handles
the concurrent one, which `ObservationConcurrencyTest` proves with eight
forked processes.

**What has no history.** Goal amounts are not observed, so FynnEdge cannot
say "you saved ₹25,000 more" — the previous value was replaced by the edit.
Page views, Twin simulations and product comparisons write nothing here:
Module 32 records explorations as decisions, in its own table, and an
attention record is not a financial fact.

### The financial plan

A plan is a reading, not a record. **There is no `financial_plans` table, no
`/financial-plan` endpoint and no plan engine.** It is three things read once
and rendered together: where the customer's figures stand, the goals they
created, and the one step the existing action ladder produced.

**`FinancialPosition`** is the factual half. It reads a `FinancialSnapshot`
and divides nothing — no ratio, no threshold and no arithmetic of its own.
What it adds is the sentence: three states (income above outgo, income
matching outgo, outgo above income) and an honest fourth for a customer who
has recorded no income at all. The same sentences reach the plan header and
the buffer rung of the ladder, because copy that describes somebody's money
is as much a source of truth as the arithmetic behind it, and two versions
of it drift.

**The buffer question can have no answer.** `Ratio::timesOver` returns zero
for a zero denominator, and reading that as a fact told a customer with
nothing recorded going out that their savings would cover less than a month.
```

`bufferIsMeasurable()` separates a low figure from an absent one, and the rung stays silent for the second.

Completion is derived, never a status. `FinancialGoal.status` stays `active` for a goal whose two amounts have met; `is_complete` is `saved >= target` at every read. A status would be a second answer that could disagree with the figures printed beside it.

`monthly_required` is a calculated reference, not a commitment. It is `remaining / months_remaining`, recomputed on every read, and there is no commitment model for it to be a commitment against. The goal card used to read "Set aside ₹9,474 a month for 19 months to get there" — an instruction the customer never agreed to, followed by a promise about where it leads. It now says what the arithmetic gives at the target and date they chose.

Nothing predicts. No "on track", no projected completion, no month by which anything will happen. `test/product_language_test.dart` scans both codebases for the phrasings, allowing a mention only where the sentence is refusing to make it.

Two rungs the planner was reaching wrongly. `exploreOptions` is documented as `goals.any and applications.empty` and its sentence reads "see what suits the goal you set" — but it checked only the applications, so a customer with no goal, and a customer whose income FynnEdge did not have, were both pointed at loan discovery. Both now match the stated rule. The negative-surplus guard from Module 33 is unchanged and still the one that matters most.

Support

NO ADVISOR SYSTEM, SUPPORT CRM, TICKETING SYSTEM OR CHAT PROVIDER IS CONNECTED. A support case is written to FynnEdge's own tables and read by whoever looks at them. Nothing is forwarded anywhere, nothing replies, and no case advances on a timer — a status describing work nobody did is a lie a customer would act on.

That is also why there is no advisor endpoint. `author_type = advisor` exists in the schema with no write path anywhere in the codebase: the model is ready for an advisor interface, and inventing an endpoint for one would be inventing the advisor.

Contextual, not generic. The point of the module is that support knows what the customer was doing. `[Get help]` on an application whose submission was never confirmed opens a request already carrying that application, and the server raises it to **high priority because of the application's state** — never because a request said so. `SupportContextType` is an allowlist (never a model class name) and `ContextResolver` looks every id up through `$user->applications()`, so another customer's id answers exactly as an id that never existed.

What a context carries is a reference, a status sentence and a product name. The status sentence is `CustomerStatus`'s own — support and the application screen cannot say two different things about the same application.

FynnEdge decides status and priority. Neither is fillable and neither is read from a request. Transitions are checked against `SupportCaseStatus::allowedNext()` before anything is written, and every move leaves an event saying who made it. A resolved case cannot be reopened.

References are `FE-SUP-YYYY-NNNNNN`, allocated by `SequentialReference` — the insert is attempted and a unique violation means take the next number. Read-then-write was measured losing six of eight concurrent requests.

Rate limits: 5 cases and 20 messages per customer per hour, both deliberately generous. A limiter that stops somebody asking for help is a worse failure than a duplicate, and duplicates are already prevented by the idempotency key.

Notifications reuse Module 29 entirely — two types, `support_resolved` and `support_waiting_for_customer`, and nothing else. A case being acknowledged is FynnEdge confirming something the customer watched happen.

Privacy. A subject, a description and a message body are the customer's

````
own words about their own money. They are never logged, never sent to
FynnAI, and never put in a notification. Logs carry the reference, category,
status and priority.

**Retention is unset.** Support history is a customer's own record of what
they asked and what they were told. No period has been decided, nothing
deletes it, and `fynn:retention` does not touch it.

## Consents

Nine types, in `App\Domain\Consent\ConsentType`. A consent is offered only
when the capability behind it exists — `credit_bureau_check`,
`document_processing`, `identity_verification` and `lender_data_sharing`
become grantable exactly when their driver is configured, and are otherwise
shown as unavailable with a reason. The ledger is append-only: a change of mind adds a record, it never
edits one.

`document_processing` and `identity_verification` are deliberately separate.
One permits sending a document somewhere to be **read**; the other permits
having an identity **checked**. A customer can reasonably want the first and
not the second.

## Tests

```bash
php artisan test           # the whole suite
./vendor/bin/pint --test   # formatting
```

The JSON shape is the contract. The Flutter models parse these keys directly
and there is a test on the app side that proves it, run against a live
server:

```bash
php artisan serve --port=8123
cd ../fynnedge && flutter test test/api_contract_test.dart \
  --dart-define=FYNN_API_BASE=http://127.0.0.1:8123
```

Run that after changing any response shape.

Standing probe suites worth extending rather than replacing: `IdorMatrixTest`
(every resource × every verb as the wrong customer), `TamperingTest` (forged
pricing, scores, extractions, consent versions), `FinancialIntegrityTest`,
`ProductionConfigTest` (what the app refuses when it believes it is in
production).

Several suites are deliberately written to fail when something is *added*
rather than when something breaks, because the failure mode being guarded
against is an absent decision rather than a wrong one:

- `OwnershipCoverageTest` reads the route table. Every authenticated route
  with a `{parameter}` must be probed as the wrong customer or carry a
  written reason for needing no probe.
- `RoleMatrixTest` holds the whole admin access matrix — 14 resources × 6
  roles — and fails if a resource exists that the matrix does not mention.
- `SensitiveExposureTest` plants sentinel values in withheld fields and
  searches the rendered HTML for them, so the guarantee is about the
  response rather than the resource definition.
- `PanelRendersTest` renders every panel page **with fixtures loaded**; an
  empty table renders almost anything.
- `VaultIsolationTest` fails if any new route serves a download.

Three tests skip where `pcntl` is unavailable. They prove concurrency
behaviour rather than simulate it, and skipping is the honest outcome; the
same races were verified manually with parallel processes.

## Operations

```bash
php artisan fynn:health                # anything stuck that needs a person
                                       # (includes urgent help requests nobody has picked up)
php artisan fynn:health --json         # same, for a monitor
php artisan fynn:retention             # what is held, and how old
````
````

```
php artisan fynn:provider-events:retry  # re-attempt events that reached nothing
php artisan queue:work                  # document reading, when QUEUE_CONNECTION=database

php artisan database:verify             # this database against the original SQLite
php artisan database:verify --allow-extra  # …tolerating rows added since
php artisan demo:purge --admin          # remove the demo customers and dev admin
```

`database:verify` answers one question: did the conversion lose anything.
It compares row counts and column shape table by table and never prints a
record's contents — it will one day be pointed at a database holding real
customers. Once demo fixtures are loaded the database legitimately holds
more than the original, which is what `--allow-extra` is for: gains are
reported as `EXTRA`, losses still fail.

`demo:purge` removes two populations: the customers `DemoFixtureSeeder`
writes, and the ones the Flutter contract test leaves behind after signing
in against a running backend. Both sets are named individually in
`App\Domain\Demo\ReservedIdentities` — never matched by prefix, so a real
customer cannot be caught by a rule getting broader. It shows what will
cascade and asks before deleting; `--dry-run` inspects without changing
anything, which is the only way to see the effect when `--force` is used
from a script. The baseline development customer is deliberately not
included: it is the one the Flutter app's mocks mirror.

`fynn:health` exits non-zero only when a customer is **blocked and cannot
resolve it themselves** — today that means a submission whose outcome was
never confirmed, where the app tells them to contact support. Everything
else is reported as attention and exits clean.

There is no monitoring stack configured in this project and none is
pretended: the exit code and `--json` are the integration points for
whatever a deployment actually has.

Environment

`.env.example` documents every variable and why it holds the value it does.
The ones that must change before a deployment faces real customers:

Variable	Development	Production	Because
`APP_ENV`	`local`	`production`	Several seeders and the log OTP sender key off it
`APP_DEBUG`	`true`	**`false`**	With debug on, every error carries file paths and a stack trace
`APP_KEY`	generated	generated	Never share one between environments
`LOG_LEVEL`	`debug`	`info`	Debug is noisy and likelier to capture something that should not be kept
`DB_CONNECTION`	`mysql`	`mysql`	SQLite is for the test suite
`DB_TIMEZONE`	`+00:00`	`+00:00`	MySQL converts `TIMESTAMP` from the session zone; unpinned, every stored time shifts
`SESSION_ENCRYPT`	`false`	**`true`**	Sessions are stored in the database, and backups copy that table
`SESSION_SECURE_COOKIE`	unset	**`true`**	Otherwise an admin session travels in clear text
`FYNNEDGE_OTP_DRIVER`	`log`	a real SMS provider	The log sender throws in production
`FYNNEDGE_OTP_DEV_HINT`	`true`	**unset**	On, the API returns the code it just issued
`TRUSTED_PROXIES`	empty	your load balancer	Untrusted, every customer presents as the proxy and the OTP limits collapse into one bucket
`FYNNEDGE_ADMIN_EMAIL` / `_PASSWORD`	unused	set, or no admin	There is no default production password

No secret belongs in the repository. `.env` is gitignored; `.env.example`
carries names and explanations, never values.

Notes for going live

- **The production checklist is the Environment table above.** None of it is
  set for you, and two of the entries — `APP_DEBUG` and
  `FYNNEDGE_OTP_DEV_HINT` — disclose information if left at their
  development values.
- **Remove the demo data.** `php artisan demo:purge --admin` takes out the
  four invented customers and the development administrator. A database that
  has been used for development otherwise carries invented loan applications
  into every report run afterwards.
- **Empty maps**: return `(object) []`, not `[]`. PHP encodes an empty
  associative array as a JSON array, which breaks strict parsers.

- **Retention periods are unset, and unset means keep.** FynnEdge does not choose one — how long a credit report, extraction, verification or provider event may be kept while consent stands is a product and legal decision. The mechanism exists: `php artisan fynn:retention` reports what is held and how old it is, and `--apply` enforces whatever periods are configured. Consent withdrawal is separate and already deletes immediately.
- **No customer document leaves FynnEdge unscanned.** `ExternalTransferGate` refuses to hand a file to a real external provider unless a malware scanner has passed it. No scanner is connected, so today that means a real provider adapter cannot ship the hole with it. Simulated providers send the bytes nowhere and are unaffected.
- **The simulated submission provider refuses to run in production.** It reports itself unavailable when `APP_ENV=production`, deliberately without an escape hatch: a customer shown a delivered application that was never sent is the worst failure this product could produce.
- **No verification expiry** is defined. `expires_at` exists and is always null: a period nobody chose would make results look stale on an arbitrary date.
- **Virus scanning** is not implemented. It is a prerequisite before any customer document is sent to an external provider.

---

7. Source D: Flutter screen-to-API flow

`fynnedge-api/docs/flutter-screen-api-flow.md`, verbatim. Written 26 Oct 2026 (`65addde`) by a Claude Code session that read the code. The repo also has a Postman collection and an OpenAPI spec served at `/docs/api`; neither is included here.

````
# FynnEdge — Flutter ↔ API screen flow

How the Flutter application talks to the Laravel API, screen by screen.

> Generated from source: `../fynnedge/lib` and `../fynnedge-api`. Every endpoint,
> screen, route and service method below was extracted from the code, not written
> from memory. Examples use reserved test numbers and invented data only.

Generated 2026-09-10.

## 1. Overview

| | |
|---|---|
| Flutter screens | 41 |
| Navigation routes (GoRouter) | 45 |
| Flutter service classes calling the API | 19 |
| Distinct API call sites in Flutter | 70 |
| Unique endpoints Flutter calls | 70 |
| Laravel API routes | 82 |
| Laravel operations (method × path) | 84 |
| Flutter calls matched to a Laravel route | 70 / 70 |
| Flutter calls with no matching route | 0 |

## 2. Architecture

```text
Screen (lib/features/**)
      │  ref.read(xProvider)
      ▼
Riverpod controller / repository
      ▼
Service (lib/data/services)
      ▼
ApiClient  ──►  Authorization: Bearer <token>
      ▼
Laravel  /api/v1/*
      ▼
Controller ──► Service ──► Model ──► MySQL
```

Providers are declared in `lib/app/providers.dart`; the screen never
constructs a service itself. `ApiClient` (`lib/data/services/api_client.dart`)
adds the bearer token, decodes the envelope and converts HTTP status codes
into typed exceptions — see [section 8](#8-error-handling).

## 3. Authentication flow

```text
SplashScreen  /            token in secure storage?
   │  no                              │ yes
   ▼                                  ▼
WelcomeScreen /welcome            GET /auth/me   ──► HomeScreen /home
   ▼
LoginScreen  /login
   │  tap Send OTP
   ▼  POST /auth/send-otp           {mobile}
OtpScreen    /otp
   │  enter code
   ▼  POST /auth/verify-otp         {mobile, reference, code, device_name}
   │  ◄── {data:{token, is_new_user, expires_at, profile}}
   ▼
token stored ──► every later call sends Authorization: Bearer <token>
   │
````
````

```
   ├─ is_new_user  ──► /onboarding/profile ──► /onboarding/intent ──► /onboarding/goal
   └─ returning    ──► /home
```

Step	Method	Endpoint	Auth	Notes
Request a code	POST	`/auth/send-otp`	public	throttled `otp-send` (3/min per number)
Verify	POST	`/auth/verify-otp`	public	throttled `otp-verify`; returns the token
Confirm a stored token	GET	`/auth/me`	bearer	used on launch
Sign out	POST	`/auth/logout`	bearer	revokes **this device's** token only

Tokens last 30 days. `dev_hint` appears in the `send-otp` response **only** in non-production builds with the development flag on; production never returns it.

4. Screen inventory

Route	Screen	File	Calls API
`/`	SplashScreen	`lib/features/splash/splash_screen.dart`	no
`/welcome`	WelcomeScreen	`lib/features/welcome/welcome_screen.dart`	no
`/login`	LoginScreen	`lib/features/auth/login_screen.dart`	yes
`/otp`	OtpScreen	`lib/features/auth/otp_screen.dart`	yes
`/onboarding/profile`	BasicProfileScreen	`lib/features/onboarding/basic_profile_screen.dart`	yes
`/onboarding/intent`	IntentScreen	`lib/features/onboarding/intent_screen.dart`	no
`/onboarding/goal`	GoalScreen	`lib/features/onboarding/goal_screen.dart`	no
`/home`	HomeScreen	`lib/features/home/home_screen.dart`	yes
`/explore`	ExploreScreen	`lib/features/explore/explore_screen.dart`	no
`/ai`	AiScreen	`lib/features/ai/ai_screen.dart`	yes
`/applications`	ApplicationsScreen	`lib/features/applications/applications_screen.dart`	yes
`/profile`	ProfileScreen	`lib/features/profile/profile_screen.dart`	yes
`/loan-discovery`	LoanDiscoveryScreen	`lib/features/loans/loan_discovery_screen.dart`	no
`/loan-options`	LoanOptionsScreen	`lib/features/loans/loan_options_screen.dart`	yes
`/loan/:id`	LoanDetailScreen	`lib/features/loans/loan_detail_screen.dart`	yes
`/apply/:id`	ApplicationReviewScreen	`lib/features/applications/application_review_screen.dart`	yes
`/fynn-twin`	TwinScreen	`lib/features/twin/twin_screen.dart`	yes
`/compare`	CompareScreen	`lib/features/loans/compare_screen.dart`	yes
`/fynn-trust/:id`	FynnTrustScreen	`lib/features/loans/fynn_trust_screen.dart`	yes
`/fynn-mirror/:id`	FynnMirrorScreen	`lib/features/loans/fynn_mirror_screen.dart`	yes
`/fynnlab`	FynnLabScreen	`lib/features/fynnlab/fynnlab_screen.dart`	no
`emi`	EmiCalculatorScreen	`lib/features/fynnlab/emi_calculator_screen.dart`	yes
`affordability`	AffordabilityScreen	`—`	no
`prepayment`	PrepaymentScreen	`—`	no
`balance-transfer`	BalanceTransferScreen	`—`	no
`emergency-fund`	EmergencyFundScreen	`—`	no
`/fynn-score`	FynnScoreScreen	`lib/features/score/fynn_score_screen.dart`	yes
`/applications/:id`	ApplicationDetailScreen	`lib/features/applications/application_detail_screen.dart`	yes
`/vault`	VaultScreen	`lib/features/vault/vault_screen.dart`	yes
`/scan`	ScanScreen	`lib/features/vault/scan_screen.dart`	yes
`/scan/:id`	FynnScanScreen	`lib/features/vault/fynn_scan_screen.dart`	yes
`/kyc/:id`	KycScreen	`lib/features/vault/kyc_screen.dart`	yes
`/second-opinion`	SecondOpinionScreen	`lib/features/vault/second_opinion_screen.dart`	no
`/financial-profile`	FinancialProfileScreen	`lib/features/profile/financial_profile_screen.dart`	yes
`/goals`	GoalsScreen	`lib/features/profile/goals_screen.dart`	yes
`/notifications`	NotificationsScreen	`lib/features/profile/notifications_screen.dart`	yes
`/credit`	CreditScreen	`lib/features/credit/credit_screen.dart`	yes
`/privacy`	PrivacyScreen	`lib/features/profile/privacy_screen.dart`	yes
`/decisions`	DecisionHistoryScreen	`lib/features/decisions/decision_history_screen.dart`	yes
`/review`	ReviewScreen	`lib/features/review/review_screen.dart`	yes
`/decisions/:id`	DecisionDetailScreen	`lib/features/decisions/decision_detail_screen.dart`	yes
`/financial-story`	FinancialTimelineScreen	`lib/features/intelligence/financial_timeline_screen.dart`	yes
`/support`	SupportScreen	`lib/features/support/support_screen.dart`	yes
`/support/new`	SupportRequestScreen	`lib/features/support/support_request_screen.dart`	yes
`/support/cases/:id`	SupportCaseScreen	`lib/features/support/support_case_screen.dart`	yes

5. Service → endpoint map

The precise mapping: every API call in the Flutter code, by the Dart method that makes it.

Customer & profile

```
| Dart method | Method | Endpoint | Laravel controller | Auth |
|---|---|---|---|---|
| `ConsentService.getConsents()` | GET | `/consents` | ConsentController@index | bearer |
| `ConsentService.grant()` | POST | `/consents` | ConsentController@store | bearer |
| `ConsentService.getHistory()` | GET | `/consents/history` | ConsentController@history | bearer |
| `ConsentService.withdraw()` | POST | `/consents/{id}/withdraw` | ConsentController@withdraw | bearer |
| `ProfileService.getFinancialProfile()` | GET | `/financial-profile` | ProfileController@financial | bearer
|
| `ProfileService.updateFinancialProfile()` | PUT | `/financial-profile` | ProfileController@updateFinancial
| bearer |
| `ProfileService.getFynnScore()` | GET | `/fynn-score` | ScoreController@show | bearer |
| `ProfileService.getGoals()` | GET | `/goals` | GoalController@index | bearer |
| `ProfileService.createGoal()` | POST | `/goals` | GoalController@store | bearer |
| `ProfileService.deleteGoal()` | DELETE | `/goals/{id}` | GoalController@destroy | bearer |
| `ProfileService.updateGoal()` | PUT | `/goals/{id}` | GoalController@update | bearer |
| `HomeService.getSnapshot()` | GET | `/home` | HomeController@index | bearer |
| `ProfileService.getProfile()` | GET | `/profile` | ProfileController@show | bearer |
| `ProfileService.updateProfile()` | PUT | `/profile` | ProfileController@update | bearer |

### Financial

| Dart method | Method | Endpoint | Laravel controller | Auth |
|---|---|---|---|---|
| `DecisionService.getDecisions()` | GET | `/decisions` | DecisionController@index | bearer |
| `DecisionService.getDecision()` | GET | `/decisions/{id}` | DecisionController@show | bearer |
| `IntelligenceService.getIntelligence()` | GET | `/intelligence` | IntelligenceController@show | bearer |
| `IntelligenceService.getTimeline()` | GET | `/intelligence/timeline` | IntelligenceController@timeline |
bearer |

### Products & matching

| Dart method | Method | Endpoint | Laravel controller | Auth |
|---|---|---|---|---|
| `MatchService.findMatches()` | GET | `/fynn-match` | MatchController@index | bearer |
| `MatchService.simulate()` | GET | `/fynn-match/{id}` | MatchController@show | bearer |
| `CatalogService.getProducts()` | GET | `/products` | ProductController@index | bearer |
| `CatalogService.getProduct()` | GET | `/products/{id}` | ProductController@show | bearer |

### Applications

| Dart method | Method | Endpoint | Laravel controller | Auth |
|---|---|---|---|---|
| `ApplicationDocumentsService.checklistFor()` | GET | `/applications/{id}/documents` |
ApplicationDocumentController@index | bearer |
| `ApplicationDocumentsService.attach()` | POST | `/applications/{id}/documents` |
ApplicationDocumentController@store | bearer |
| `ApplicationDocumentsService.detach()` | DELETE | `/applications/{id}/documents/{id}` |
ApplicationDocumentController@destroy | bearer |
| `ApplicationDocumentsService.review()` | POST | `/applications/{id}/documents/{id}/review` |
ApplicationDocumentController@review | bearer |
| `SubmissionService.statusFor()` | GET | `/applications/{id}/submission` |
ApplicationSubmissionController@show | bearer |
| `SubmissionService.submit()` | POST | `/applications/{id}/submission` |
ApplicationSubmissionController@store | bearer |

### Documents & verification

| Dart method | Method | Endpoint | Laravel controller | Auth |
|---|---|---|---|---|
| `CreditService.check()` | POST | `/credit/check` | CreditController@check | bearer |
| `CreditService.getStatus()` | GET | `/credit/status` | CreditController@status | bearer |
| `KycService.statusFor()` | GET | `/documents/{id}/kyc` | KycController@show | bearer |
| `KycService.verify()` | POST | `/documents/{id}/kyc` | KycController@store | bearer |
| `KycService.resolve()` | POST | `/documents/{id}/kyc/resolve` | KycController@resolve | bearer |
| `DocumentScanService.statusFor()` | GET | `/documents/{id}/scan` | DocumentScanController@show | bearer |
| `DocumentScanService.scan()` | POST | `/documents/{id}/scan` | DocumentScanController@store | bearer |
| `DocumentScanService.review()` | POST | `/documents/{id}/scan/review` | DocumentScanController@review |
bearer |

### Notifications

| Dart method | Method | Endpoint | Laravel controller | Auth |
|---|---|---|---|---|
| `NotificationService.getPreferences()` | GET | `/notifications/preferences` |
```

```
NotificationPreferenceController@show | bearer |
| `NotificationService.updatePreferences()` | PUT | `/notifications/preferences` |
NotificationPreferenceController@update | bearer |
| `NotificationService.markAllRead()` | POST | `/notifications/read-all` |
NotificationController@markAllRead | bearer |
| `NotificationService.getUnreadCount()` | GET | `/notifications/unread-count` |
NotificationController@unreadCount | bearer |
| `NotificationService.markRead()` | POST | `/notifications/{id}/read` | NotificationController@markRead |
bearer |

### Support

| Dart method | Method | Endpoint | Laravel controller | Auth |
|---|---|---|---|---|
| `SupportService.getCases()` | GET | `/support/cases` | SupportController@index | bearer |
| `SupportService.createCase()` | POST | `/support/cases` | SupportController@store | bearer |
| `SupportService.getCase()` | GET | `/support/cases/{id}` | SupportController@show | bearer |
| `SupportService.closeCase()` | POST | `/support/cases/{id}/close` | SupportController@close | bearer |
| `SupportService.addMessage()` | POST | `/support/cases/{id}/messages` | SupportController@addMessage |
bearer |
| `SupportService.getCategories()` | GET | `/support/categories` | SupportController@categories | bearer |

### FynnAI

| Dart method | Method | Endpoint | Laravel controller | Auth |
|---|---|---|---|---|
| `AiReply.getConversations()` | GET | `/ai/conversations` | AiController@index | bearer |
| `AiReply.startConversation()` | POST | `/ai/conversations` | AiController@store | bearer |
| `AiReply.deleteConversation()` | DELETE | `/ai/conversations/{id}` | AiController@destroy | bearer |
| `AiReply.getConversation()` | GET | `/ai/conversations/{id}` | AiController@show | bearer |
| `AiReply.send()` | POST | `/ai/conversations/{id}/messages` | AiController@send | bearer |

### Other

| Dart method | Method | Endpoint | Laravel controller | Auth |
|---|---|---|---|---|
| `ApplicationDraft.getApplications()` | GET | `/applications` | ApplicationController@index | bearer |
| `ApplicationDraft.getApplication()` | GET | `/applications/{id}` | ApplicationController@show | bearer |
| `ApplicationDraft.create()` | POST | `/applications` | ApplicationController@store | bearer |
| `ApplicationDraft.update()` | PUT | `/applications/{id}` | ApplicationController@update | bearer |
| `ApplicationDraft.submit()` | POST | `/applications/{id}/submit` | ApplicationController@submit | bearer |
| `ApplicationDraft.cancel()` | POST | `/applications/{id}/cancel` | ApplicationController@cancel | bearer |
| `OtpChallenge.sendOtp()` | POST | `/auth/send-otp` | AuthController@sendOtp | public |
| `OtpChallenge.verifyOtp()` | POST | `/auth/verify-otp` | AuthController@verifyOtp | public |
| `OtpChallenge.logout()` | POST | `/auth/logout` | AuthController@logout | bearer |
| `OtpChallenge.currentUser()` | GET | `/auth/me` | AuthController@me | bearer |
| `DocumentUpload.getDocuments()` | GET | `/documents` | DocumentController@index | bearer |
| `DocumentUpload.getDocument()` | GET | `/documents/{id}` | DocumentController@show | bearer |
| `DocumentUpload.getLimits()` | GET | `/documents/limits` | DocumentController@limits | bearer |
| `DocumentUpload.upload()` | POST | `/documents` | DocumentController@store | bearer |
| `DocumentUpload.download()` | GET | `/documents/{id}/download` | DocumentController@download | bearer |
| `DocumentUpload.delete()` | DELETE | `/documents/{id}` | DocumentController@destroy | bearer |
| `TwinCapabilities.capabilities()` | GET | `/fynn-twin` | FynnTwinController@index | bearer |
| `TwinCapabilities.simulate()` | POST | `/fynn-twin/simulate` | FynnTwinController@simulate | bearer |

## 6. Screen → API matrix

Endpoints reachable from each screen, resolved through its controllers and the
providers they read. A screen that shares a controller with another will show
that controller's endpoints too — this is a reachability map, not a per-tap trace.

| Screen | Route | Controllers | Endpoints reachable |
|---|---|---|---|
| SplashScreen | `/` | — | 0 |
| WelcomeScreen | `/welcome` | — | 0 |
| LoginScreen | `/login` | — | 4 |
| OtpScreen | `/otp` | — | 4 |
| BasicProfileScreen | `/onboarding/profile` | `home_controller.dart` | 10 |
| IntentScreen | `/onboarding/intent` | — | 0 |
| GoalScreen | `/onboarding/goal` | — | 0 |
| HomeScreen | `/home` | `intelligence_controller.dart`, `notification_controller.dart`,
`home_controller.dart` | 27 |
| ExploreScreen | `/explore` | — | 0 |
| AiScreen | `/ai` | `consent_controller.dart`, `ai_controller.dart` | 30 |
```

```
| ApplicationsScreen | `/applications` | `application_controller.dart` | 6 |
| ProfileScreen | `/profile` | `home_controller.dart`, `profile_controller.dart` | 10 |
| LoanDiscoveryScreen | `/loan-discovery` | — | 0 |
| LoanOptionsScreen | `/loan-options` | `loan_controller.dart` | 2 |
| LoanDetailScreen | `/loan/:id` | `loan_controller.dart` | 2 |
| ApplicationReviewScreen | `/apply/:id` | `loan_controller.dart`, `application_controller.dart` | 8 |
| TwinScreen | `/fynn-twin` | `loan_controller.dart`, `twin_controller.dart` | 4 |
| CompareScreen | `/compare` | `loan_controller.dart` | 2 |
| FynnTrustScreen | `/fynn-trust/:id` | `loan_controller.dart` | 2 |
| FynnMirrorScreen | `/fynn-mirror/:id` | `profile_controller.dart`, `loan_controller.dart` | 11 |
| FynnLabScreen | `/fynnlab` | — | 0 |
| EmiCalculatorScreen | `emi` | `profile_controller.dart` | 9 |
| FynnScoreScreen | `/fynn-score` | `profile_controller.dart` | 9 |
| ApplicationDetailScreen | `/applications/:id` | `application_controller.dart` | 6 |
| VaultScreen | `/vault` | `vault_controller.dart` | 6 |
| ScanScreen | `/scan` | `add_document_controller.dart`, `vault_controller.dart` | 9 |
| FynnScanScreen | `/scan/:id` | `scan_controller.dart`, `vault_controller.dart` | 9 |
| KycScreen | `/kyc/:id` | `kyc_controller.dart`, `vault_controller.dart` | 9 |
| SecondOpinionScreen | `/second-opinion` | — | 0 |
| FinancialProfileScreen | `/financial-profile` | `financial_profile_controller.dart` | 9 |
| GoalsScreen | `/goals` | `intelligence_controller.dart`, `goals_controller.dart` | 35 |
| NotificationsScreen | `/notifications` | `notification_controller.dart` | 26 |
| CreditScreen | `/credit` | `credit_controller.dart` | 2 |
| PrivacyScreen | `/privacy` | `consent_controller.dart` | 4 |
| DecisionHistoryScreen | `/decisions` | `decision_controller.dart` | 26 |
| ReviewScreen | `/review` | `decision_controller.dart`, `intelligence_controller.dart` | 26 |
| DecisionDetailScreen | `/decisions/:id` | `decision_controller.dart` | 26 |
| FinancialTimelineScreen | `/financial-story` | `intelligence_controller.dart` | 26 |
| SupportScreen | `/support` | `support_controller.dart` | 26 |
| SupportRequestScreen | `/support/new` | `support_controller.dart` | 26 |
| SupportCaseScreen | `/support/cases/:id` | `support_controller.dart` | 26 |

## 7. API → screen matrix

Which Flutter code calls each endpoint — read this before changing one.

| Method | Endpoint | Called by | Laravel controller | Throttle |
|---|---|---|---|---|
| DELETE | `/ai/conversations/{id}` | `AiReply.deleteConversation()` | AiController@destroy | api |
| DELETE | `/applications/{id}/documents/{id}` | `ApplicationDocumentsService.detach()` |
ApplicationDocumentController@destroy | api |
| DELETE | `/documents/{id}` | `DocumentUpload.delete()` | DocumentController@destroy | api, vault-write |
| DELETE | `/goals/{id}` | `ProfileService.deleteGoal()` | GoalController@destroy | api |
| GET | `/ai/conversations` | `AiReply.getConversations()` | AiController@index | api |
| GET | `/ai/conversations/{id}` | `AiReply.getConversation()` | AiController@show | api |
| GET | `/applications` | `ApplicationDraft.getApplications()` | ApplicationController@index | api |
| GET | `/applications/{id}` | `ApplicationDraft.getApplication()` | ApplicationController@show | api |
| GET | `/applications/{id}/documents` | `ApplicationDocumentsService.checklistFor()` |
ApplicationDocumentController@index | api |
| GET | `/applications/{id}/submission` | `SubmissionService.statusFor()` |
ApplicationSubmissionController@show | api |
| GET | `/auth/me` | `OtpChallenge.currentUser()` | AuthController@me | api |
| GET | `/consents` | `ConsentService.getConsents()` | ConsentController@index | api |
| GET | `/consents/history` | `ConsentService.getHistory()` | ConsentController@history | api |
| GET | `/credit/status` | `CreditService.getStatus()` | CreditController@status | api |
| GET | `/decisions` | `DecisionService.getDecisions()` | DecisionController@index | api |
| GET | `/decisions/{id}` | `DecisionService.getDecision()` | DecisionController@show | api |
| GET | `/documents` | `DocumentUpload.getDocuments()` | DocumentController@index | api |
| GET | `/documents/limits` | `DocumentUpload.getLimits()` | DocumentController@limits | api |
| GET | `/documents/{id}` | `DocumentUpload.getDocument()` | DocumentController@show | api |
| GET | `/documents/{id}/download` | `DocumentUpload.download()` | DocumentController@download | api |
| GET | `/documents/{id}/kyc` | `KycService.statusFor()` | KycController@show | api |
| GET | `/documents/{id}/scan` | `DocumentScanService.statusFor()` | DocumentScanController@show | api |
| GET | `/financial-profile` | `ProfileService.getFinancialProfile()` | ProfileController@financial | api |
| GET | `/fynn-match` | `MatchService.findMatches()` | MatchController@index | api |
| GET | `/fynn-match/{id}` | `MatchService.simulate()` | MatchController@show | api |
| GET | `/fynn-score` | `ProfileService.getFynnScore()` | ScoreController@show | api |
| GET | `/fynn-twin` | `TwinCapabilities.capabilities()` | FynnTwinController@index | api |
| GET | `/goals` | `ProfileService.getGoals()` | GoalController@index | api |
| GET | `/home` | `HomeService.getSnapshot()` | HomeController@index | api |
| GET | `/intelligence` | `IntelligenceService.getIntelligence()` | IntelligenceController@show | api |
| GET | `/intelligence/timeline` | `IntelligenceService.getTimeline()` | IntelligenceController@timeline |
api |
| GET | `/notifications/preferences` | `NotificationService.getPreferences()` |
```

```
NotificationPreferenceController@show | api |
| GET | `/notifications/unread-count` | `NotificationService.getUnreadCount()` |
NotificationController@unreadCount | api |
| GET | `/products` | `CatalogService.getProducts()` | ProductController@index | api |
| GET | `/products/{id}` | `CatalogService.getProduct()` | ProductController@show | api |
| GET | `/profile` | `ProfileService.getProfile()` | ProfileController@show | api |
| GET | `/support/cases` | `SupportService.getCases()` | SupportController@index | api |
| GET | `/support/cases/{id}` | `SupportService.getCase()` | SupportController@show | api |
| GET | `/support/categories` | `SupportService.getCategories()` | SupportController@categories | api |
| POST | `/ai/conversations` | `AiReply.startConversation()` | AiController@store | api, ai-write |
| POST | `/ai/conversations/{id}/messages` | `AiReply.send()` | AiController@send | api, ai-write |
| POST | `/applications` | `ApplicationDraft.create()` | ApplicationController@store | api |
| POST | `/applications/{id}/cancel` | `ApplicationDraft.cancel()` | ApplicationController@cancel | api |
| POST | `/applications/{id}/documents` | `ApplicationDocumentsService.attach()` |
ApplicationDocumentController@store | api |
| POST | `/applications/{id}/documents/{id}/review` | `ApplicationDocumentsService.review()` |
ApplicationDocumentController@review | api |
| POST | `/applications/{id}/submission` | `SubmissionService.submit()` |
ApplicationSubmissionController@store | api, lender-submit |
| POST | `/applications/{id}/submit` | `ApplicationDraft.submit()` | ApplicationController@submit | api |
| POST | `/auth/logout` | `OtpChallenge.logout()` | AuthController@logout | api |
| POST | `/auth/send-otp` | `OtpChallenge.sendOtp()` | AuthController@sendOtp | otp-send |
| POST | `/auth/verify-otp` | `OtpChallenge.verifyOtp()` | AuthController@verifyOtp | otp-verify |
| POST | `/consents` | `ConsentService.grant()` | ConsentController@store | api |
| POST | `/consents/{id}/withdraw` | `ConsentService.withdraw()` | ConsentController@withdraw | api |
| POST | `/credit/check` | `CreditService.check()` | CreditController@check | api, credit-check |
| POST | `/documents` | `DocumentUpload.upload()` | DocumentController@store | api, vault-write |
| POST | `/documents/{id}/kyc` | `KycService.verify()` | KycController@store | api, kyc-write |
| POST | `/documents/{id}/kyc/resolve` | `KycService.resolve()` | KycController@resolve | api |
| POST | `/documents/{id}/scan` | `DocumentScanService.scan()` | DocumentScanController@store | api, scan-
write |
| POST | `/documents/{id}/scan/review` | `DocumentScanService.review()` | DocumentScanController@review |
api |
| POST | `/fynn-twin/simulate` | `TwinCapabilities.simulate()` | FynnTwinController@simulate | api |
| POST | `/goals` | `ProfileService.createGoal()` | GoalController@store | api |
| POST | `/notifications/read-all` | `NotificationService.markAllRead()` |
NotificationController@markAllRead | api |
| POST | `/notifications/{id}/read` | `NotificationService.markRead()` | NotificationController@markRead |
api |
| POST | `/support/cases` | `SupportService.createCase()` | SupportController@store | api, support-create |
| POST | `/support/cases/{id}/close` | `SupportService.closeCase()` | SupportController@close | api |
| POST | `/support/cases/{id}/messages` | `SupportService.addMessage()` | SupportController@addMessage |
api, support-message |
| PUT | `/applications/{id}` | `ApplicationDraft.update()` | ApplicationController@update | api |
| PUT | `/financial-profile` | `ProfileService.updateFinancialProfile()` | ProfileController@updateFinancial
| api |
| PUT | `/goals/{id}` | `ProfileService.updateGoal()` | GoalController@update | api |
| PUT | `/notifications/preferences` | `NotificationService.updatePreferences()` |
NotificationPreferenceController@update | api |
| PUT | `/profile` | `ProfileService.updateProfile()` | ProfileController@update | api |

## 8. Error handling

`ApiClient._send()` converts status codes into typed exceptions. Screens catch
these and render a message; the mapping is exact and lives in one file.

| Status | Flutter exception | What it means | Typical UI |
|---|---|---|---|
| 401 | `UnauthorizedException` | token missing, expired or revoked | session cleared, returns to `/login` |
| 403 | `PermissionRequiredException` | a consent has not been given | prompts for the consent |
| 404 | `NotFoundException` | not found **or not yours** — the API does not distinguish | "not found" state
|
| 422 | `ValidationException` | field errors, with `errors{field:[…]}` | inline field messages via
`forField()` |
| 429 | `RateLimitedException` | throttled; honours `Retry-After` | countdown, resend disabled |
| 501 | `CapabilityUnavailableException` | no provider connected for that capability | explains the feature
is unavailable |
| 502/503/504 | `UpstreamUnavailableException` | provider reachable but failing | retry prompt |
| ≥500 | `ServerException` | server fault | generic failure |
| — | `NetworkException` | timeout or no connection | offline state |

The **404-not-403** behaviour is deliberate on the API side: a record belonging
to another customer is indistinguishable from one that does not exist, so an
attacker cannot probe for existence. Flutter treats both the same way.
```

````
## 9. Documents

| Screen | Action | Method | Endpoint | Notes |
|---|---|---|---|---|
| VaultScreen `/vault` | list | GET | `/documents` | |
| VaultScreen | limits | GET | `/documents/limits` | drives client-side validation before upload |
| VaultScreen | upload | POST | `/documents` | `multipart/form-data`, field `file` + `type` + `name` |
| VaultScreen | open | GET | `/documents/{id}` | metadata |
| VaultScreen | download | GET | `/documents/{id}/download` | streamed bytes, authenticated every request |
| VaultScreen | delete | DELETE | `/documents/{id}` | |
| ScanScreen `/scan/{id}` | read a document | GET/POST | `/documents/{id}/scan` | |
| KycScreen `/kyc/{id}` | verify identity | GET/POST | `/documents/{id}/kyc` | |

Limits come from the API rather than being hard-coded in the app: 10 MB, and
PDF/JPEG/PNG/HEIC/WebP. File content is sniffed server-side, so a renamed
executable is rejected. There is no public or signed URL — bytes only ever
travel through an authenticated request.

## 10. Endpoints Flutter does not call

14 of the 84 Laravel operations are not called by the app today.
Unused is not unnecessary — several are alternate verbs on endpoints the app does use.

| Method | Endpoint | Note |
|---|---|---|
| DELETE | `/ai/memory` | not currently used by the app |
| DELETE | `/ai/memory/{param}` | not currently used by the app |
| DELETE | `/notifications/devices/{param}` | not currently used by the app |
| GET | `/ai/context` | not currently used by the app |
| GET | `/ai/memory` | not currently used by the app |
| GET | `/financial-summary` | not currently used by the app |
| GET | `/goals/{param}` | not currently used by the app |
| GET | `/notifications` | not currently used by the app |
| GET | `/notifications/{param}` | not currently used by the app |
| PATCH | `/applications/{param}` | alternate verb; the app uses PUT |
| PATCH | `/financial-profile` | alternate verb; the app uses PUT |
| POST | `/compare` | not currently used by the app |
| POST | `/notifications/devices` | not currently used by the app |
| POST | `/provider-events/{param}` | provider ingress — not a customer endpoint |

## 11. Journeys

### Applying for a loan

```text
HomeScreen /home                     GET /home
   │ tap Explore
   ▼
LoanDiscoveryScreen /loan-discovery  GET /products
   ▼ enter amount + tenure
LoanOptionsScreen /loan-options      GET /fynn-match?amount&tenure_months
   ▼ pick a product
LoanDetailScreen /loan/:id           GET /fynn-match/{id}
   ▼ Apply
ApplicationReviewScreen /apply/:id   POST /applications
   ▼
ApplicationDetailScreen /applications/:id
   ├─ GET  /applications/{id}
   ├─ GET  /applications/{id}/documents      checklist
   ├─ POST /applications/{id}/documents      attach from the vault
   ├─ POST /applications/{id}/submit         complete in FynnEdge
   └─ POST /applications/{id}/submission     send to a provider → 501 today
```

`status` on the application drives what the screen offers. The values come from
`App\Domain\Application\ApplicationStatus`: `started`, `submitted`,
`under_review`, `action_needed`, `approved`, `declined`, `disbursed`,
`cancelled`. The resource also returns `can_submit`, `can_cancel` and
`is_editable`, so the app does not re-derive the rules.

### Support

```text
````
````

```
SupportScreen /support               GET /support/cases · GET /support/categories
   ▼ Get help
SupportRequestScreen /support/new    POST /support/cases   (Idempotency via idempotency_key)
   ▼
SupportCaseScreen /support/cases/:id GET /support/cases/{id}
   ├─ POST /support/cases/{id}/messages
   └─ POST /support/cases/{id}/close
```

Vault and verification

```text
VaultScreen /vault      GET /documents · GET /documents/limits
   ├─ upload            POST /documents          multipart
   ├─ download          GET  /documents/{id}/download
   ├─ ScanScreen /scan/:id   GET/POST /documents/{id}/scan   → 501 (no provider)
   └─ KycScreen  /kyc/:id    GET/POST /documents/{id}/kyc    → 501 (no provider)
```

12. State that drives navigation

API	Field	Values	Decides
`GET /auth/me`	token valid?	200 / 401	splash → home, or → login
`POST /auth/verify-otp`	`is_new_user`	true / false	onboarding, or straight to home
`GET /profile`	`onboarding_complete`	true / false	whether onboarding is offered
`GET /applications/{id}`	`status`	the 8 `ApplicationStatus` values	which actions the detail screen shows
`GET /applications/{id}`	`can_submit`, `can_cancel`, `is_editable`	boolean	button availability
`GET /consents`	`outstanding_required`	list	whether a consent must be requested first
`GET /credit/status`	`status`	`not_configured` …	whether a credit check can be offered

13. Capabilities with no provider connected

Five capabilities answer **501** with a machine-readable `reason`. The app has a dedicated exception for this and explains the feature is unavailable rather than showing a failure.

Capability	Endpoint	`reason`
Credit bureau	`POST /credit/check`	`not_configured`
Lender submission	`POST /applications/{id}/submission`	`lender_provider_unavailable`
Identity verification	`POST /documents/{id}/kyc`	provider unavailable
Document reading	`POST /documents/{id}/scan`	provider unavailable
Push delivery	notification devices	no push driver

These refusal paths are tested. The **success** paths cannot be verified until a provider is connected.

14. Testing

Suite	Where	Result
Flutter contract test	`../fynnedge/test/api_contract_test.dart`	113/113
Laravel suite	`php artisan test`	see the phase report
API ownership (IDOR)	`tests/Feature/Api/IdorMatrixTest.php`, `OwnershipCoverageTest.php`	pass
Rate limits	`tests/Feature/Api/RateLimitCoverageTest.php`	pass

Run the contract test against a local backend:

```bash
php artisan serve --port=8123
cd ../fynnedge && flutter test test/api_contract_test.dart \
  --dart-define=FYNN_API_BASE=http://127.0.0.1:8123
```

15. Mismatches

Flutter makes 70 distinct endpoint calls. Every one resolves to a Laravel route — **0 unmatched**. No endpoint is called that the API does not serve, and no call uses a method the route does not accept.

16. Reference

```
- OpenAPI: `/docs/api` (UI) and `/docs/api.json` — 82 documented operations
- API guide: `README.md`, section *The API*
- Route list: `php artisan route:list --path=api/v1`
```

End of pack.